

PROVA DE APTIDÃO PROFISSIONAL

COMÉRCIOLOCAL



CURSO PROFISSIONAL DE TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

NOME ALUNO: Vladimiro Pankratau

N.º: 25

PROFESSOR ORIENTADOR: Luís Fernandes

ANO LETIVO: 2025/2026



AGRADECIMENTOS

Quero deixar o meu agradecimento aos professores Ângelo Mateus e Luís Fernandes, por todo o tempo disponibilizado, sempre com a maior atenção possível. Estiveram sempre disponíveis para atenderem as nossas dificuldades, sem nos deixarem que algo faltasse.



ÍNDICE

Agradecimentos	I
Índice	II
Índice de figuras	V
Índice de tabelas	VIII
Lista de siglas e acrónimos	IX
1. Introdução	10
1.1. Fundamentação	11
1.2. Descrição do projeto	12
1.3. Objetivos do projeto	13
1.4. Estado do conhecimento	14
2. Planificação do projeto	16
2.1. Calendarização	16
2.1.1. Calendarização prevista	16
2.1.2. Calendarização realizada	18
2.2. Tecnologia utilizada	19
2.2.1. Ambiente de Desenvolvimento	19
2.2.2. Backend	21
2.2.3. Frontend	22
2.2.4. Serviços de Terceiros	23
2.2.5. Ferramentas de controlo de versão	24
2.2.6. Ferramenta de apoio ao desenvolvimento	25
2.2.7. Extensões do Visual Studio Code	26
2.3. Protótipo	27
2.3.1. Página Inicial (Landing Page)	27
2.3.2. Páginas de Autenticação (Login e Registo)	28
2.3.3. Página de Exploração e Visualização de Lojas	29
2.3.4. Página do Lojista no Perfil da sua Loja	29
2.3.5. Página de Edição do Perfil da Loja	30
2.3.6. Página Administrativa (Dashboard)	31



2.4. Base de Dados	32
2.4.1. Enquadramento Tecnológico	32
2.4.2. Listagem de Entidades (Tabelas)	34
2.4.3. Descrição de Cada Tabela	34
2.4.4. Relacionamentos entre Tabelas (Chaves Estrangeiras)	37
2.4.5. Integridade Referencial e Ações em Cascata	38
2.4.6. Povoamento Inicial (Seeders)	38
3. Implementação do projeto	41
3.1. Configuração do Ambiente e Criação do Projeto	41
3.1.1. Instalação do Laragon	41
3.1.2. Instalação e Configuração do PHP	41
3.1.3. Instalação do Composer	44
3.1.4. Instalação do Node.js	44
3.1.5. Criação do Repositório no GitHub	44
3.1.6. Clonagem do Repositório	45
3.1.7. Criação do Projeto Laravel	45
3.1.8. Configuração da Base de Dados	47
3.1.9. Configuração do Mailtrap	48
3.1.10. Inicialização do Servidor	50
3.2. Estrutura de pastas do Laravel	50
3.2.1. Pasta app	51
3.2.2. Pasta bootstrap	51
3.2.3. Pasta config	52
3.2.4. Pasta database	52
3.2.5. Pasta public	53
3.2.6. Pasta resources	53
3.2.7. Pasta routes	54
3.2.8. Pasta storage	54
3.2.9. Pasta tests	55
3.2.10. Pasta vendor	55
3.2.11. Ficheiros na Raiz do Projeto	55

3.3. Implementação da Autenticação	56
3.3.1. Criação das Rotas de Autenticação	56
3.3.2. Página de Login	57
3.3.3. Página de Registo de Conta	58
3.3.4. Validação e criação da conta no controlador	59
3.3.5. Diferença entre visitante e lojista após o registo	59
3.3.6. Processo de Login e Redirecionamento por Tipo de Utilizador	60
3.4. Criação da Loja	60
3.5. Upload da Foto de Perfil da Loja	62
3.6. Aprovação da Loja pelo Administrador	64
3.7. Gestão de Publicações por Parte do Lojista	65
3.8. Denúncias por Parte do Utilizador Visitante	66
3.9. Explorar Lojas e Publicações	68
4. Problemas e soluções encontradas	70
4.1. Gestão de Armazenamento para Publicações de Vídeo	70
4.2. Experiência de Login do Lojista após Rejeição do Pedido de Conta	72
4.3. Erro de Sintaxe na Página Administrativa de Gestão de Lojas	74
4.4. Controllers com muito código	76
5. Conclusão	78
5.1. Cumprimento dos Objetivos	78
5.2. Perspetivas de Evolução Futura	78
5.3. Autoavaliação	78
Referências bibliográficas	80

ÍNDICE DE FIGURAS

Figura 1 – Logotipo do Laragon	19
Figura 2 - Logotipo do Visual Studio Code	20
Figura 3 - Logotipo do Composer	20
Figura 4 - Logotipo do Node.js	20
Figura 5 - Logotipo do PHP	21
Figura 6 - Logotipo do Laravel	21
Figura 7 - Logotipo do MySQL Workbench	22
Figura 8 - Logotipo do HTML5	22
Figura 9 - Logotipo do CSS3	22
Figura 10 - Logotipo do JavaScript	23
Figura 11 - Logotipo do Mailtrap	23
Figura 12 - Logotipo do Pixabay	24
Figura 13 - Logotipo do Magnific	24
Figura 14 - Logotipo do GitHub	25
Figura 15 - Logotipo do GitKraken	25
Figura 16 - Logotipo do GitHub Copilot	26
Figura 17 - Landing Page	28
Figura 18 – Página de Login	28
Figura 19 – Página de Registo	28
Figura 20 – Página Explorar	29
Figura 21 - Visitante a visualizar uma loja	29
Figura 22 - Lojista no perfil da sua loja	30
Figura 23 - Lojista a editar o perfil da sua loja	31
Figura 24 - Página administrativa (Dashboard)	32
Figura 25 - Diagrama Entidade-Relação da base de dados do ComércioLocal	33
Figura 26 – Pasta php-8.5.4-Win32-vs17-x64 no diretório do Laragon	42
Figura 27 – Definição da nova versão instalada do PHP no Laragon	42
Figura 28 – Abertura do ficheiro php.ini através do Laragon	43
Figura 29 – Localização da palavra "zip" e remoção do ";" na linha 813	43
Figura 30 - Repositório ComercioLocal no GitHub	44
Figura 31 - Clone do repositório no GitKraken	45
Figura 32 - Comando composer global require laravel/installer	45
Figura 33 - Comando laravel new ComercioLocal	45
Figura 34 - Comando laravel new ComercioLocal	46
Figura 35 - Configuração do laravel new ComercioLocal	46
Figura 36 - Comandos cd ComercioLocal e npm install	47
Figura 37 - Comando npm run build	47
Figura 38 - Inicialização do MySQL no Laragon	47

Figura 39 - Criação da base de dados comerciocal no MySQL Workbench	47
Figura 40 - Configuração da ligação à base de dados comerciocal no ficheiro .env	48
Figura 41 - Sandbox comerciocal no Mailtrap.....	48
Figura 42 - Código de configuração do Mailtrap para Laravel 9+	49
Figura 43 - Configuração do Mailtrap no ficheiro .env	49
Figura 44 - Inicialização do servidor com ph partisan serve	50
Figura 45 - Página inicial padrão do Laravel	50
Figura 46 - Conteúdo da pasta app.....	51
Figura 47 - Conteúdo da pasta bootstrap.....	51
Figura 48 - Conteúdo da pasta config	52
Figura 49 - Conteúdo da pasta database.....	52
Figura 50 - Conteúdo da pasta public.....	53
Figura 51 - Conteúdo da pasta public/storage.....	53
Figura 52 - Conteúdo da pasta resources	54
Figura 53 - Conteúdo da pasta resources/views	54
Figura 54 - Conteúdo da pasta routes	54
Figura 55 - Conteúdo da pasta storage.....	55
Figura 56 - Conteúdo da pasta storage/app/public	55
Figura 57 - Ficheiros e pastas na raiz do projeto.....	56
Figura 58 - código das rotas de autenticação	57
Figura 59 - página com o formulário de login	57
Figura 60 - Excerto do código que corresponde ao formulário de Login.....	58
Figura 61 - página com o formulário de registo	58
Figura 62 - Excerto do código que corresponde ao formulário de Registo	59
Figura 63 - primeiro excerto do controlador com a lógica de registo	59
Figura 64 - excerto da lógica de redireccionamento após o registo.....	60
Figura 65 - Excerto do código da lógica principal de login	60
Figura 66 - página com o formulário de registo de Loja.....	61
Figura 67 - rotas protegidas.....	61
Figura 68 - Excerto da lógica de criação da loja no controlador	61
Figura 69 - página com a zona para inserir uma fotografia para o perfil da Loja	62
Figura 70 - excerto das rotas do upload da foto de perfil	62
Figura 71 - Excerto do controlador com o processamento e gravação da foto	63
Figura 72 - página com os dois botões de decisão das denúncias	64
Figura 73 - código dos botões de decisão da secção de pedidos pendentes no painel do administrador	64
Figura 74 - rotas de aprovação e rejeição da loja.....	64
Figura 75 - código do controlador com a aprovação e rejeição da loja	65
Figura 76 - página com as funcionalidades para gerir as publicações	66
Figura 77 - código do controlador para eliminar uma publicação	66

Figura 78 - página com o formulário de denúncia de uma Loja.....	67
Figura 79 - rotas de funcionalidade de denúncias	67
Figura 80 – Excerto do código do controlador para registar uma denúncia	68
Figura 81 - Página com o formulário de filtros e a zona que gera os cartões das Lojas.....	68
Figura 82 – Excerto do método que aplica filtros, ordenação e retorna a vista	69
Figura 83 - Botão para escolher a opção de vídeo por ligação externa	71
Figura 84 - Zona de input para inserir o URL do vídeo	71
Figura 85 - Código de validação do campo de URL para vídeo	71
Figura 86 - Vídeo incorporado com o player do YouTube.....	71
Figura 87 - Vídeo reproduzido com o player nativo do navegador (URL com extensão .mp4)	71
Figura 88 - Exemplo de vídeo do YouTube com o player da plataforma	71
Figura 89 - Botão para rejeitar a loja no painel administrativo	72
Figura 90 - Função para rejeitar o pedido e marcar a loja para eliminação	73
Figura 91 - Detecção do indicador de eliminação e apresentação da mensagem ao lojista	73
Figura 92 - Exibição da mensagem de erro na página de login	74
Figura 93 - Mensagem de erro apresentada pelo Laravel.....	75
Figura 94 - Código Blade que estava a causar o erro de sintaxe.....	75
Figura 95 - Código Blade corrigido que solucionou o problema.....	76
Figura 96 - Exemplo da dimensão elevada de um ficheiro de controlador	76
Figura 97 - Exemplo da organização dos controladores da pasta Loja	77

ÍNDICE DE TABELAS

Tabela 1 - Cronograma inicial	17
Tabela 2 - Cronograma final	19

LISTA DE SIGLAS E ACRÓNIMOS

PAP – Prova de Aptidão Profissional

KFC – Kentucky Fried Chicken

TGPSI – Técnico de Gestão e Programação de Sistemas Informáticos

AHRESP – Associação da Hotelaria, Restauração e Similares de Portugal

PHP – HyperText Preprocessor

MVC – Model-View-Controller

npm – Node Package Manager

HTML – HyperText Markup Language

CSS – Cascading Style Sheets

JS – JavaScript

JSON – JavaScript Object Notation

HTTP – HyperText Transfer Protocol

SQL – Structured Query Language

EER – Enhanced Entity-Relationship

DER – Diagrama Entidade-Relacionamento

1. INTRODUÇÃO

A PAP consiste na apresentação e defesa, perante um júri, de um projeto consubstanciado num produto, material ou intelectual, numa intervenção ou numa atuação, consoante a natureza dos cursos, bem como no respetivo relatório final de realização e apreciação crítica, demonstrativo de conhecimentos, aptidões, atitudes e competências profissionais adquiridas ao longo do percurso formativo do aluno, em todas as componentes de formação, com especial enfoque nas áreas de competência inscritas no Perfil dos Alunos à Saída da Escolaridade Obrigatória e no perfil profissional associado à respetiva qualificação.¹

O presente relatório relata as diversas etapas do desenvolvimento do projeto realizado no âmbito da PAP, encontrando-se estruturado em cinco capítulos. O primeiro capítulo, a presente introdução, apresenta as motivações e o contexto que levaram ao desenvolvimento do projeto, bem como a sua descrição geral, fundamentação e objetivos. O segundo capítulo, Planificação do Projeto, descreve o processo de planeamento, incluindo a calendarização prevista e realizada, as ferramentas utilizadas, o protótipo e a estrutura da base de dados. O terceiro capítulo, Implementação do Projeto, detalha todo o processo de configuração do ambiente de desenvolvimento, a criação do projeto Laravel, a estrutura de pastas da aplicação e a implementação das principais funcionalidades, como a autenticação e o registo de lojas. O quarto capítulo, Problemas e Soluções Encontradas, documenta os principais obstáculos técnicos enfrentados ao longo do desenvolvimento, bem como as soluções adotadas para os ultrapassar. O quinto e último capítulo, Conclusão, apresenta uma reflexão final sobre o trabalho realizado, o cumprimento dos objetivos definidos e uma autoavaliação do desempenho pessoal ao longo do projeto.

Nos últimos anos, o avanço das tecnologias digitais tem vindo a transformar significativamente a forma como as pessoas consomem e interagem com o comércio. As plataformas online assumem um papel cada vez mais relevante na divulgação de produtos e serviços, permitindo uma maior proximidade entre comerciantes e consumidores. No entanto, muitos estabelecimentos locais continuam a enfrentar dificuldades na adaptação a este novo contexto digital, seja por falta de recursos, conhecimento técnico ou tempo para investir em estratégias de promoção online.

O projeto ComércioLocal surge precisamente com o intuito de responder a esta necessidade, criando uma plataforma digital dedicada à promoção do comércio tradicional das Caldas da Rainha. Através de um website intuitivo e de fácil utilização, pretende-se reunir num só espaço os diversos estabelecimentos locais, oferecendo-lhes uma ferramenta eficaz para divulgar os seus produtos e serviços, enquanto se facilita o acesso da população e dos visitantes a uma oferta comercial diversificada e autêntica.

¹ Portaria n.º 2235-A/2018 de 23 de agosto, Artigo 29.º, Ponto 2

Desta forma, o projeto pretende não apenas apoiar o desenvolvimento económico local, mas também valorizar a identidade e o dinamismo do comércio das Caldas da Rainha, contribuindo para a sua modernização e sustentabilidade no contexto digital atual.

Importa ainda salientar, desde já, que o ComércioLocal se dirige exclusivamente ao comércio tradicional e independente do concelho, não sendo admitidos estabelecimentos pertencentes a cadeias de franchising ou grandes superfícies comerciais, tais como McDonald's, KFC, Burger King, Pingo Doce, Worten, Radio Popular, entre outros. Esta opção visa proteger o propósito do projeto, promover negócios de raiz local, de gestão familiar ou autónoma, que constituem a verdadeira identidade comercial das Caldas da Rainha, bem como evitar a desvirtuação do diretório com marcas que já dispõem, por si só, de elevada visibilidade e capacidade de marketing digital.

1.1. FUNDAMENTAÇÃO

A ideia para o projeto ComércioLocal emergiu da observação direta da realidade do comércio nas Caldas da Rainha, nomeadamente da constatação de que muitos pequenos e médios estabelecimentos não possuem qualquer presença digital ou, quando possuem, esta se encontra fragmentada, desatualizada ou alojada exclusivamente em redes sociais de difícil pesquisa e organização. Durante o percurso formativo no curso TGPSI, foi possível perceber o potencial das tecnologias web para colmatar lacunas de visibilidade e acessibilidade. O contacto com comerciantes locais, em conversas informais e visitas a feiras e mercados, revelou uma procura latente por soluções simples, económicas e centralizadas que lhes permitissem divulgar as suas atividades sem dependerem de conhecimentos avançados de informática. Assim, o projeto nasceu da interseção entre uma necessidade social real, as competências técnicas adquiridas em programação, bases de dados e design de interfaces, e o desejo de contribuir ativamente para a comunidade local.

A conceção do projeto "ComércioLocal" revela-se uma mais-valia significativa, justificando-se plenamente a sua execução por diversas ordens de razão, que importa detalhar.

Em primeiro lugar, o projeto apresenta uma clara utilidade pública e económica. Ao facilitar o acesso dos consumidores ao comércio local, promove-se ativamente a economia circular, fortalecendo o tecido empresarial da região. Simultaneamente, o "ComércioLocal" atua como um agente redutor da assimetria digital que frequentemente favorece as grandes superfícies em detrimento dos pequenos negócios de bairro, nivelando o campo de jogo e oferecendo a estes últimos uma presença online competitiva e acessível.

Em segundo lugar, o projeto distingue-se pela baixa barreira de entrada que oferece aos comerciantes. Ao invés da criação de websites individuais, um processo geralmente oneroso e que exige conhecimentos técnicos especializados, o "ComércioLocal" propõe uma plataforma unificada e intuitiva. Neste modelo, o comerciante vê a sua participação simplificada ao máximo, necessitando apenas de fornecer os seus dados institucionais e comerciais para, de seguida, gerir o seu perfil através

de um painel de controlo de fácil utilização, isentando-o, assim, de quaisquer conhecimentos de programação.

Em terceiro lugar, o projeto assenta numa forte valorização da identidade local. Ao concentrar o seu âmbito de atuação exclusivamente nas Caldas da Rainha, o "ComércioLocal" não se limita a ser um mero diretório; transcende essa função ao criar um ponto de encontro digital que celebra a história, a cultura e os produtos típicos da região. Desta forma, o projeto estabelece uma ponte valiosa entre residentes, turistas e a oferta comercial mais autêntica do concelho, reforçando o seu carácter único.

Por fim, importa destacar a escalabilidade e sustentabilidade técnica do projeto. Desenvolvido com recurso a tecnologias modernas, cujos detalhes serão aprofundados nos capítulos técnicos deste relatório, o site foi concebido para garantir uma fácil manutenção e uma evolução futura descomplexada. Esta arquitetura robusta permite a integração gradual de novas funcionalidades, como sistemas de georreferenciação, avaliações de clientes ou até a implementação de um sistema de encomendas online, assegurando, simultaneamente, um baixo custo de alojamento, o que contribui para a viabilidade e longevidade do projeto.

1.2. DESCRIÇÃO DO PROJETO

A aplicação desenvolvida, designada "ComércioLocal", assume-se como uma plataforma web de consulta e divulgação do tecido comercial do concelho das Caldas da Rainha, estruturada em torno de quatro níveis de acesso distintos, cada um com permissões e funcionalidades especificamente delineadas para responder às necessidades dos diferentes intervenientes no ecossistema digital.

No primeiro nível, encontra-se o utilizador não autenticado, ou visitante anónimo. Qualquer pessoa que aceda ao endereço do website sem efetuar login vê o seu acesso limitado à landing page (página de entrada). Esta página, de carácter estático e institucional, apresenta os objetivos do projeto, o seu âmbito geográfico (concelho das Caldas da Rainha) e as vantagens para comerciantes e consumidores, integrando ainda chamadas à ação que incentivam o registo ou o início de sessão. Salienta-se que o visitante anónimo não possui, em circunstância alguma, a capacidade de visualizar a listagem de estabelecimentos, pesquisar lojas, aceder a páginas de detalhe ou interagir com qualquer conteúdo dinâmico da plataforma, funcionando esta página como uma mera porta de entrada informativa.

Num segundo patamar, surge o utilizador visitante (autenticado). Após criar uma conta e iniciar sessão com as suas credenciais (endereço de correio eletrónico e palavra-passe), o utilizador comum vê o seu leque de possibilidades ampliado substancialmente. Este perfil permite-lhe explorar a totalidade da plataforma, pesquisar estabelecimentos por nome ou categoria, visualizar páginas de detalhe com informações como morada, contacto, horário, descrição e galeria multimédia, e usufruir de um conjunto de funcionalidades personalizadas. Destacam-se, neste âmbito, a possibilidade de

guardar lojas como favoritas, classificar publicações ou conteúdos específicos como interessantes e submeter denúncias de conteúdos ou estabelecimentos que considere inadequados ou violadores das regras da plataforma, conferindo-lhe um papel ativo na curadoria comunitária do diretório.

O terceiro nível de acesso é reservado ao utilizador comerciante (autenticado). Mediante registo prévio e aprovação da conta por parte do administrador, o proprietário de um estabelecimento obtém permissão para se autenticar na plataforma. A partir desse momento, dispõe de um painel de controlo privado que lhe faculta a inserção, edição e eliminação de conteúdos relativos, exclusivamente, ao seu próprio negócio, incluindo imagens, vídeos, textos descritivos, horários de funcionamento e dados de contacto. Esta arquitetura de permissões garante que apenas o comerciante detém a capacidade de gerir a informação da sua loja, impedindo, de forma inequívoca, alterações ou interferências por parte de terceiros.

Por fim, o quarto nível corresponde ao utilizador administrador (autenticado). Para além das funcionalidades inerentes ao perfil de comerciante, o administrador usufrui de uma *dashboard* administrativa que lhe concede poderes acrescidos. Através desta interface, pode aprovar novos registos de estabelecimentos, analisar e deliberar sobre as denúncias submetidas pelos visitantes, bem como bloquear, eliminar ou reativar lojas, publicações ou utilizadores sempre que se verifiquem infrações às diretrizes da plataforma, assegurando, desta forma, a integridade e a qualidade do ecossistema digital.

Importa ainda esclarecer, a título de nota final, que o *website* não integra funcionalidades de comunicação direta entre visitantes e comerciantes (como mensagens privadas ou *chat*), nem permite a realização de transações comerciais ou compras *online*. O projeto assume-se, assim, como um diretório digital de natureza informativa e promocional, focado na valorização e na divulgação do comércio tradicional das Caldas da Rainha, onde a autenticação constitui um requisito obrigatório para qualquer interação com os conteúdos dinâmicos da plataforma.

1.3. OBJETIVOS DO PROJETO

O projeto ComércioLocal foi concebido com o propósito de atingir um conjunto de objetivos gerais e específicos, os quais traduzem as suas mais-valias fundamentais e conferem sentido à sua execução. Estes objetivos podem ser agrupados em três dimensões principais: uma dimensão social e económica, uma dimensão funcional e técnica, e uma dimensão formativa e de sustentabilidade.

Num primeiro plano, o projeto visa centralizar a informação do comércio local, eliminando a dispersão atualmente existente, onde os dados dos estabelecimentos se encontram fragmentados por múltiplas redes sociais, sites desatualizados ou, simplesmente, em cartazes físicos afixados nas montras. Ao oferecer um único ponto de referência digital fidedigno, o ComércioLocal pretende aumentar a visibilidade digital dos pequenos negócios, permitindo que qualquer estabelecimento, mesmo sem competências tecnológicas, passe a ser encontrado online através de pesquisa simples por categoria, nome ou localização, nivelando, assim, o campo de jogo face às grandes superfícies.

Paralelamente, o projeto ambiciona facilitar a descoberta de negócios por parte dos consumidores, oferecendo uma experiência de pesquisa intuitiva, rápida e filtrável, que permita a residentes e turistas encontrar, por exemplo, "um talho aberto ao sábado à tarde nas Caldas" ou "uma ourivesaria com reparação de relógios". Desta forma, ao tornar os pequenos negócios mais acessíveis, o ComércioLocal contribui ativamente para promover a economia local e o comércio de proximidade, incentivando o consumo no concelho e reduzindo a fuga de compras para grandes superfícies ou plataformas online externas, fortalecendo, assim, o tecido empresarial da região.

Por fim, nesta dimensão, o projeto assume um compromisso com a valorização da identidade e autenticidade do comércio caldense. Ao criar uma plataforma dedicada exclusivamente a esta região, o ComércioLocal contribui para a construção de uma imagem de marca coesa, onde a tradição e a modernidade coexistem, celebrando a história, a cultura e os produtos típicos do concelho.

No plano funcional, o projeto destaca-se por fornecer uma ferramenta de gestão simples para comerciantes. O painel de controlo desenvolvido permite que cada comerciante insira, altere ou remova os seus próprios dados sem necessidade de conhecimentos técnicos, reduzindo a sua dependência de terceiros e conferindo-lhe autonomia plena na gestão da sua presença digital.

Numa perspetiva formativa, o ComércioLocal foi também idealizado para servir como projeto de síntese das competências TGPSI, demonstrando, de forma integrada, conhecimentos de programação (*frontend* e *backend*), modelação e gestão de bases de dados, design de interfaces, usabilidade, segurança informática e gestão de projetos, constituindo, assim, um reflexo do percurso formativo do aluno.

Por último, o projeto foi arquitetado para ser sustentável e facilmente extensível, garantindo a sua viabilidade a longo prazo. A arquitetura modular adotada permite que, futuramente, sejam adicionadas funcionalidades como encomendas online, sistema de reservas, georreferenciação avançada, notificações por correio eletrónico ou integração com sistemas municipais, assegurando que a plataforma possa evoluir em sintonia com as necessidades do comércio local e com o avanço tecnológico.

1.4. ESTADO DO CONHECIMENTO

Antes do início do desenvolvimento do projeto ComércioLocal, o conhecimento existente sobre as tecnologias e ferramentas que viriam a ser utilizadas já era considerável, embora não exaustivo. Ao longo do percurso formativo no curso TGPSI, foi possível adquirir uma base sólida em várias das tecnologias que seriam empregues na construção da plataforma, o que permitiu uma abordagem mais confiante e eficiente durante a fase de implementação.

Do ponto de vista técnico, o contacto prévio com o framework Laravel revelou-se particularmente relevante. Embora o projeto representasse o primeiro desenvolvimento de maior envergadura utilizando esta tecnologia, já existiam conhecimentos fundamentais sobre a sua

estrutura, o padrão MVC (Model-View-Controller), o sistema de rotas, as views em Blade e a interação com bases de dados através do Eloquent ORM. Da mesma forma, a linguagem PHP, que serve de base ao Laravel, já era familiar, tendo sido utilizada em projetos anteriores ao longo do curso, o que facilitou a compreensão da lógica backend da aplicação.

No que diz respeito à gestão de bases de dados, o conhecimento do MySQL e da ferramenta MySQL Workbench já estava consolidado, tendo sido adquirido em unidades curriculares anteriores. Este conhecimento revelou-se essencial para a modelação da base de dados do projeto, a definição das relações entre tabelas e a criação das migrações e seeders que suportam a estrutura de dados da plataforma.

No campo do desenvolvimento frontend, já existiam competências em HTML, CSS e JavaScript, bem como alguma experiência com o motor de templates Blade do Laravel. Embora a complexidade da interface do ComércioLocal tenha exigido um aprofundamento destes conhecimentos, a base existente permitiu um arranque mais rápido e uma maior autonomia na construção das páginas e componentes visuais da aplicação.

Para além das competências técnicas, o projeto beneficiou também de um conhecimento prévio sobre o domínio do problema. O contacto com a realidade do comércio local das Caldas da Rainha, aliado à observação direta das dinâmicas dos pequenos negócios de bairro, permitiu uma compreensão mais clara das necessidades dos comerciantes e dos consumidores. Esta familiaridade com o contexto local foi fundamental para definir os requisitos funcionais da plataforma, nomeadamente a estrutura de categorias de lojas, os tipos de conteúdo a disponibilizar e a simplicidade necessária nos fluxos de utilização, de modo a garantir que a ferramenta fosse acessível mesmo a comerciantes com poucos recursos tecnológicos.

Desta forma, o estado do conhecimento inicial revelou-se adequado para enfrentar os desafios do projeto, tendo sido possível construir sobre uma base sólida de competências técnicas e de compreensão do domínio, ao mesmo tempo que se consolidaram e expandiram esses conhecimentos ao longo de todo o processo de desenvolvimento.

2. PLANIFICAÇÃO DO PROJETO

A planificação do projeto constituiu uma etapa fundamental para o sucesso do desenvolvimento do ComércioLocal, uma vez que permitiu organizar de forma estruturada todas as atividades inerentes à conceção, implementação e validação da plataforma. Neste âmbito, foram definidas as etapas do projeto, os prazos para cada uma delas e os objetivos específicos a atingir em cada fase. Paralelamente, procedeu-se à seleção das tecnologias a adotar, bem como ao planeamento da arquitetura da aplicação, assegurando que todos os componentes se encontravam devidamente alinhados com os requisitos funcionais e não funcionais identificados.

As secções seguintes apresentam, de forma detalhada, a calendarização do projeto (com a distinção entre o previsto e o realizado), as ferramentas e tecnologias utilizadas ao longo do desenvolvimento, o protótipo da aplicação e a estrutura da base de dados que suporta o funcionamento da plataforma.

2.1. CALENDARIZAÇÃO

A primeira etapa do processo de planificação consistiu na elaboração de um cronograma que contemplasse todas as fases do projeto, desde a fase de conceção inicial até à entrega final, tendo em consideração a data estipulada para a conclusão e apresentação do mesmo. Este cronograma serviu como guia orientador ao longo de todo o ano letivo, permitindo uma gestão eficaz do tempo e dos recursos disponíveis.

2.1.1. CALENDARIZAÇÃO PREVISTA

A calendarização prevista foi elaborada no início do desenvolvimento da PAP, numa fase em que o projeto se encontrava ainda em estado conceptual. Nesta altura, o cronograma traduzia uma visão macro e indicativa das principais etapas a serem desenvolvidas ao longo do ano, servindo como um roteiro inicial para orientar o trabalho. Embora baseada em estimativas e suposições, esta planificação preliminar revelou-se crucial para estabelecer metas intermédias e para identificar, desde cedo, os marcos temporais mais relevantes do projeto, designadamente a conclusão da análise de requisitos, o desenvolvimento do protótipo, a implementação do *backend* e do *frontend*, a integração da base de dados e a realização dos testes finais.

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Definição do tema									
Definição das tecnologias									
Pesquisa sobre o tema									
Esboço visual do projeto									
Desenvolvimento da base de dados									
Desenvolvimento do frontend (páginas web estáticas)									
Ligação da base de dados ao projeto com Laravel									
Registo e login de utilizadores									
Lojistas criarem, eliminarem e editarem publicações									
Visitantes denunciarem lojas e publicações									

Tabela 1 - Cronograma inicial

2.1.2. CALENDARIZAÇÃO REALIZADA

No término do projeto, foi elaborada a calendarização real, que documenta, de forma organizada e sintética, as datas em que cada um dos passos cruciais do desenvolvimento foi efetivamente concluído. Esta calendarização reflete o percurso efetivo do projeto, evidenciando as tarefas realizadas, a sua sequência temporal e a duração de cada uma delas. A comparação entre a calendarização prevista e a realizada permite identificar desvios, compreender as razões que os motivaram (como imprevistos técnicos, ajustes de requisitos ou necessidade de aprofundamento de determinadas matérias) e extrair lições para futuros projetos, constituindo, assim, um instrumento de reflexão e de melhoria contínua.

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Definição do tema									
Definição das tecnologias									
Pesquisa sobre o tema									
Esboço da base de dados									
Esboço visual do projeto									
Desenvolvimento do frontend (páginas web estáticas)									
Desenvolvimento da base de dados									
Ligação da base de dados ao projeto com Laravel									
Registo e login de utilizadores									
Definição e proteção de rotas									
Lojistas criarem, eliminarem e editarem publicações									
Visitantes adicionarem lojas e publicações aos favoritos									
Visitantes adicionarem lojas aos não interessados									
Lojistas afixarem publicações ao topo do seu perfil									

Visitantes denunciarem lojas e publicações									
Administrador autorizar a criação da conta de lojistas									
Administrador ativar, bloquear, eliminar publicações e lojas									

Tabela 2 - Cronograma final

2.2. TECNOLOGIA UTILIZADA

No desenvolvimento do projeto, foram utilizadas diversas tecnologias, linguagens e ferramentas, integradas num ambiente de desenvolvimento local. A escolha de cada componente teve em consideração os requisitos funcionais do sistema e as boas práticas de desenvolvimento de software.

2.2.1. AMBIENTE DE DESENVOLVIMENTO

Laragon

O Laragon foi utilizado como ambiente de desenvolvimento local completo, integrando servidor web (Apache), MySQL e PHP. Permitiu criar um ambiente isolado e portátil, com virtual host personalizado para o projeto, facilitando os testes em diferentes páginas e rotas da aplicação.

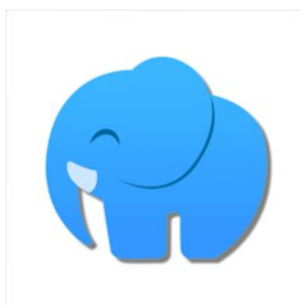


Figura 1 – Logotipo do Laragon

Visual Studio Code

O Visual Studio Code foi o editor de código fonte utilizado ao longo de todo o ciclo de desenvolvimento. Forneceu funcionalidades como realce de sintaxe, preenchimento automático de código, terminal integrado e suporte a extensões que potenciaram a produtividade na escrita de código PHP, HTML, CSS e JavaScript.



Figura 2 - Logotipo do Visual Studio Code

Composer

O Composer foi utilizado como gestor de dependências para PHP. Através do ficheiro `composer.json`, foram geridas as bibliotecas necessárias ao projeto, com destaque para o framework Laravel e extensões adicionais.

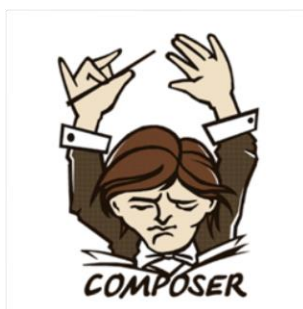


Figura 3 - Logotipo do Composer

Node.js (npm e Vite)

O Node.js foi instalado exclusivamente como ferramenta de build para os ativos de frontend, sem intervenção na lógica de backend. Através do gestor de pacotes npm, foram instaladas as dependências necessárias, com destaque para o Vite, que compilou e serviu os ficheiros CSS e JavaScript em tempo real durante o desenvolvimento.



Figura 4 - Logotipo do Node.js

2.2.2. BACKEND

PHP

O PHP é a linguagem de programação do lado do servidor que constituiu a base do *backend* do projeto. Sobre o PHP assentou todo o *framework* Laravel, sendo responsável pelo processamento de rotas, execução de controladores, interação com a base de dados, geração dinâmica de vistas, validação de dados, gestão de sessões e autenticação.



Figura 5 - Logotipo do PHP

Laravel

O Laravel foi o *framework* PHP escolhido para estruturar o *backend* do site. A sua arquitetura MVC (Model-View-Controller) permitiu separar claramente a lógica de negócio, os dados e a apresentação. Foram utilizados modelos para representar as tabelas da base de dados, controladores para processar os pedidos HTTP e vistas com o motor de *templates* Blade para gerar o HTML dinâmico.

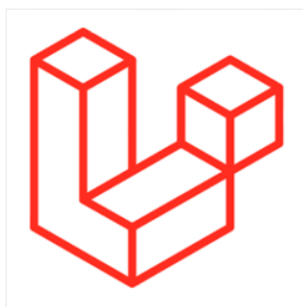


Figura 6 - Logotipo do Laravel

MySQL Workbench

O MySQL Workbench foi utilizado para desenhar, visualizar e gerir a base de dados relacional do projeto. Foram realizadas tarefas como modelação conceptual (diagramas EER), execução de scripts SQL e monitorização de consultas.



Figura 7 - Logotipo do MySQL Workbench

2.2.3. FRONTEND

HTML5

O HTML foi utilizado para estruturar semanticamente o conteúdo de cada página do site, através de elementos como cabeçalho, navegação, secções principais, formulários e rodapé.



Figura 8 - Logotipo do HTML5

CSS3

O CSS foi aplicado para definir a apresentação visual de cada página, incluindo layout, cores, tipografia, espaçamentos e responsividade (adaptação a diferentes tamanhos de ecrã).



Figura 9 - Logotipo do CSS3

JavaScript

O JavaScript foi utilizado para adicionar interatividade ao site, incluindo validação de formulários do lado do cliente, filtros em tempo real, confirmação de ações e menu responsivo para dispositivos móveis.



Figura 10 - Logotipo do JavaScript

2.2.4. SERVIÇOS DE TERCEIROS

Mailtrap

O Mailtrap foi integrado no projeto como serviço de simulação de envio de e-mails em ambiente de desenvolvimento, evitando o envio acidental de mensagens reais. Foi utilizado para testar funcionalidades como formulários de contacto, registo de novos utilizadores e recuperação de palavra-passe.

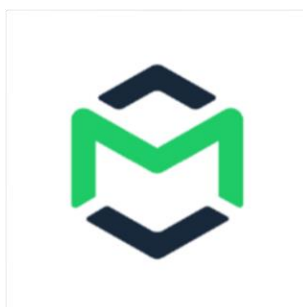


Figura 11 - Logotipo do Mailtrap

Pixabay

O Pixabay foi utilizado como fonte de vídeos livres de direitos de autor para integrar no projeto. Estes vídeos serviram como conteúdo de exemplo para as publicações dos lojistas, permitindo simular a componente multimédia da plataforma sem incorrer em violações de direitos de autor. A utilização deste serviço garantiu que o projeto apresentasse conteúdo visual rico e diversificado, essencial para demonstrar a funcionalidade de partilha de vídeos.



Figura 12 - Logotipo do Pixabay

Magnific

O Magnific foi a plataforma escolhida para a obtenção de imagens livres de direitos de autor. As imagens provenientes deste serviço foram utilizadas como exemplos de fotografias de perfil das lojas e como conteúdo ilustrativo nas publicações dos lojistas. A qualidade e variedade das imagens disponíveis permitiu criar um ambiente visualmente apelativo e realista para o projeto, demonstrando as capacidades da plataforma sem comprometer a legalidade dos recursos utilizados.



Figura 13 - Logotipo do Magnific

2.2.5. FERRAMENTAS DE CONTROLO DE VERSÃO

GitHub

O GitHub foi utilizado como plataforma de alojamento remoto do repositório do projeto. Para além de garantir a segurança e backup do código fonte, o GitHub permitiu manter um histórico completo de todas as alterações realizadas ao longo do desenvolvimento. A plataforma serviu ainda como ponto central de partilha e colaboração, facilitando o acesso ao código a partir de diferentes ambientes e garantindo a rastreabilidade de cada modificação efetuada.



Figura 14 - Logotipo do GitHub

GitKraken

O GitKraken foi a ferramenta cliente Git utilizada para gerir o controlo de versões do projeto. Através da sua interface gráfica intuitiva, foi possível visualizar o histórico de commits, criar e gerir ramos (branches), efetuar *merges* e resolver conflitos de forma simplificada. O GitKraken permitiu manter um registo organizado e detalhado de cada etapa do desenvolvimento, registando os *commits* correspondentes a cada funcionalidade, correção ou melhoria implementada no projeto.

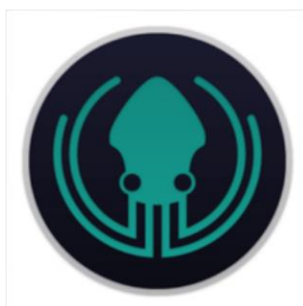


Figura 15 - Logotipo do GitKraken

2.2.6. FERRAMENTA DE APOIO AO DESENVOLVIMENTO

GitHub Copilot

O GitHub Copilot foi utilizado como assistente de programação baseado em inteligência artificial. Esta ferramenta auxiliou no desenvolvimento de *templates* estáticos em HTML, CSS e JavaScript, sugerindo blocos de código, estruturas de layout e estilos de forma contextual. O Copilot contribuiu para acelerar o processo de desenvolvimento, especialmente na criação de componentes visuais repetitivos e na sugestão de soluções para problemas comuns de *frontend*, permitindo uma maior concentração na lógica específica do projeto.

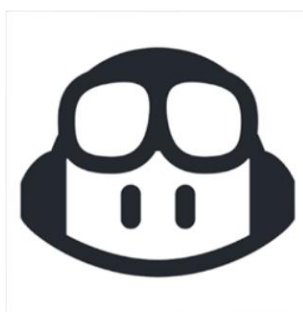


Figura 16 - Logotipo do GitHub Copilot

2.2.7. EXTENSÕES DO VISUAL STUDIO CODE

Para potenciar a produtividade e facilitar o desenvolvimento, foram instaladas as seguintes extensões no Visual Studio Code:

- Dracula Theme Official – tema de cores para o editor, melhorando a legibilidade do código.
- Format HTML in PHP – formatação correta de código HTML dentro de ficheiros PHP.
- HTML CSS Support – autocompletamento para classes e IDs CSS em ficheiros HTML e Blade.
- Image preview – visualização de miniaturas de imagens diretamente no editor.
- Live Server – servidor local com recarregamento automático para testes rápidos de frontend.
- Material Icon Theme – ícones distintos para diferentes tipos de ficheiros e pastas.
- PHP Debug – integração do depurador XDebug para breakpoints e inspeção de variáveis.
- PHP Extension Pack – pacote com extensões essenciais para desenvolvimento em PHP.
- PHP Intelephense – análise de código estática, autocompletamento avançado e navegação entre símbolos.
- PHP IntelliSense – autocompletamento e documentação para PHP. PHP Server – servidor PHP embutido para testes isolados.

2.3. PROTÓTIPO

No âmbito do desenvolvimento de qualquer projeto de software, a fase de prototipagem assume particular relevância, na medida em que permite materializar, de forma visual e interativa, as ideias conceituais iniciais, antecipando a estrutura, o fluxo de navegação e as funcionalidades da aplicação antes do início da codificação propriamente dita. Para o projeto ComércioLocal, foram elaborados *mockups* correspondentes às principais páginas da plataforma, os quais serviram como guia orientador durante todo o processo de desenvolvimento, permitindo validar a adequação das soluções propostas e identificar, previamente, os requisitos funcionais e de interface que o projeto teria de cumprir.

As imagens que se seguem documentam esses *mockups* iniciais, evidenciando a evolução conceptual entre a fase de idealização e a implementação final, bem como as alterações e melhorias entretanto introduzidas.

2.3.1. PÁGINA INICIAL (LANDING PAGE)

A Figura 17 representa a página inicial do projeto, correspondente à landing page. Esta é a primeira interface com que o utilizador se depara ao aceder ao endereço do website, assumindo, por isso, um papel crucial na apresentação institucional do projeto. A página foi concebida para exibir informação estática sobre o ComércioLocal, designadamente a génese da ideia, os objetivos gerais e as mais-valias para comerciantes e consumidores. De realçar que, no protótipo inicial, foi prevista uma secção de contactos, integrando um formulário que permitiria aos visitantes submeter questões e esclarecer dúvidas acerca da plataforma, funcionalidade que, embora não implementada na versão final, evidencia a preocupação inicial com a comunicação com os utilizadores.

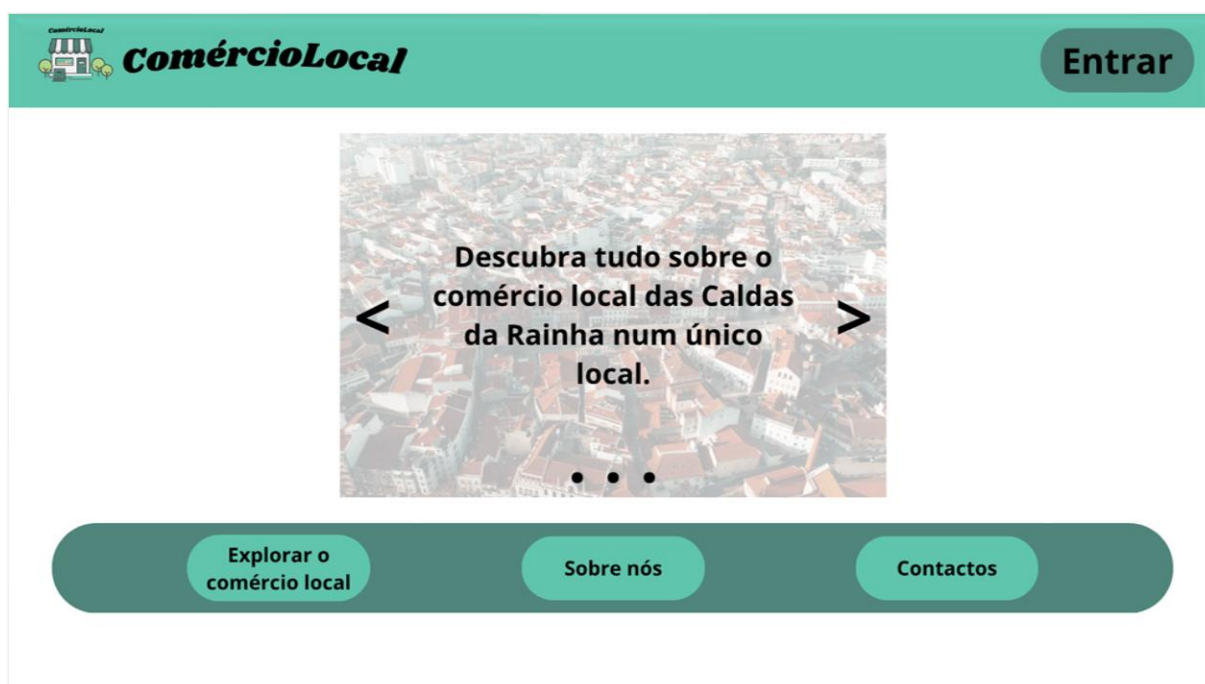


Figura 17 - Landing Page

2.3.2. PÁGINAS DE AUTENTICAÇÃO (LOGIN E REGISTO)

As Figuras 18 e 19 ilustram, respetivamente, as páginas de início de sessão (login) e de registo de novos utilizadores. O formulário de autenticação manteve, na sua essência, a estrutura prevista no protótipo, tendo sido preservado na versão final do projeto. Já o formulário de registo sofreu alterações significativas, uma vez que, na implementação final, o processo de registo passou a ser condicionado pelo tipo de perfil pretendido (utilizador comum ou comerciante), o que implicou a criação de múltiplos formulários distribuídos por mais do que uma página, de modo a acomodar a informação específica exigida para cada tipo de conta.

Figura 18 – Página de Login

Figura 19 – Página de Registo

2.3.3. PÁGINA DE EXPLORAÇÃO E VISUALIZAÇÃO DE LOJAS

As Figuras 20 e 21 representam, sucessivamente, a página de exploração de estabelecimentos e a página de detalhe de uma loja, na perspetiva de um utilizador visitante autenticado. Desde a fase de conceção inicial, foi prevista a inclusão de um menu de filtros, com o objetivo de proporcionar uma pesquisa mais eficaz e direcionada, permitindo aos utilizadores refinar os resultados por categoria, nome ou localização. Paralelamente, foi também idealizado, desde o início, o sistema de denúncias, que confere aos utilizadores a capacidade de assinalar conteúdos ou estabelecimentos que considerem inadequados, contribuindo para a moderação comunitária da plataforma.

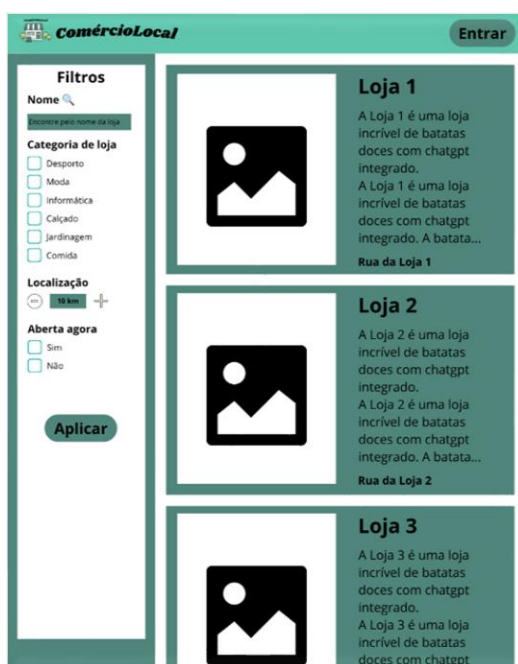


Figura 20 – Página Explorar

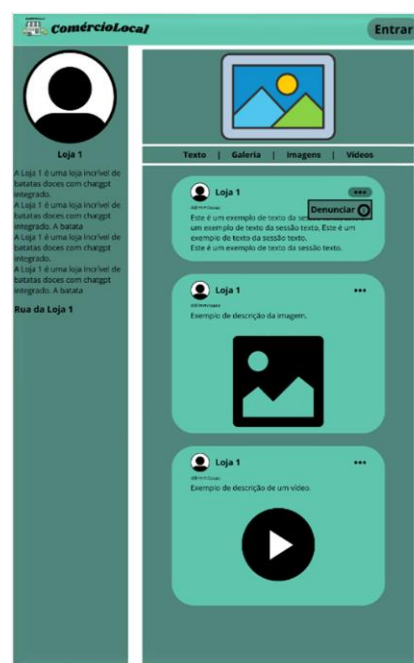


Figura 21 - Visitante a visualizar uma loja

2.3.4. PÁGINA DO LOJISTA NO PERFIL DA SUA LOJA

A Figura 22 diz respeito à página do lojista no perfil da sua própria loja, ou seja, à vista que o comerciante autenticado tem do seu estabelecimento. No protótipo inicial, foi equacionada a possibilidade de exibir as publicações organizadas por tipologia de conteúdo (texto, fotografia e vídeo), numa lógica de separação tabular. Contudo, esta abordagem não foi implementada na versão final, tendo-se optado por uma apresentação cronológica e unificada dos conteúdos. As restantes funcionalidades previstas foram, no entanto, integradas, tendo sido objeto de aperfeiçoamento e expansão ao longo do desenvolvimento.

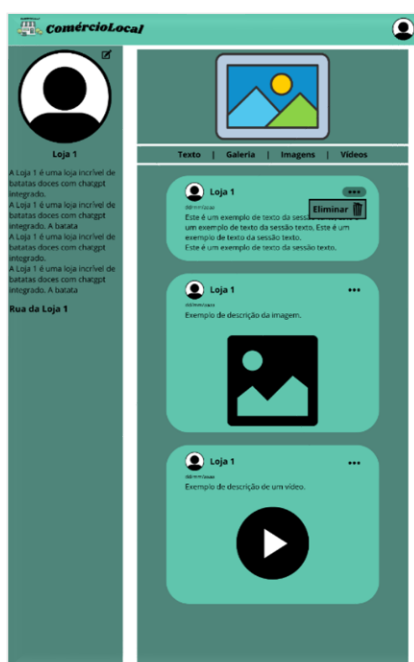


Figura 22 - Lojista no perfil da sua loja

2.3.5. PÁGINA DE EDIÇÃO DO PERFIL DA LOJA

A Figura 23 apresenta o mockup da página destinada à edição do perfil da loja, numa perspetiva do comerciante. Na fase de prototipagem, a ideia inicial consistiu em implementar o formulário de edição como uma área sobreposta à página do perfil da loja, funcionando como um elemento modal. Todavia, na implementação final, optou-se por criar uma página independente e dedicada exclusivamente a esta função, proporcionando uma experiência de utilização mais clara e intuitiva, com maior espaço para a disposição dos campos do formulário.

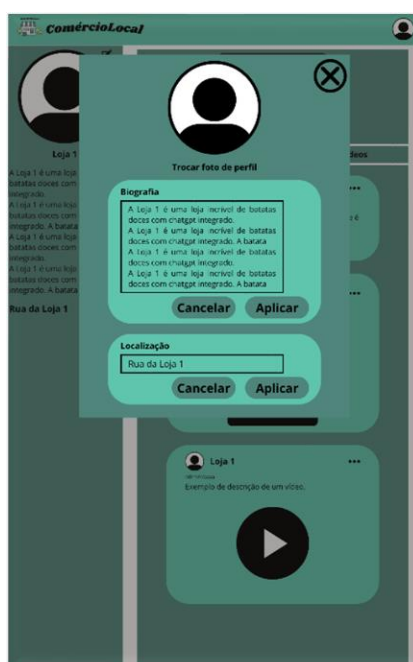


Figura 23 - Lojista a editar o perfil da sua loja

2.3.6. PÁGINA ADMINISTRATIVA (DASHBOARD)

Por fim, a Figura 24 documenta o mockup da página de gestão administrativa, reservada ao perfil de administrador. Esta foi, de todas as áreas da plataforma, aquela que sofreu as alterações mais profundas em relação ao esboço visual inicial. O protótipo primitivo revelou-se excessivamente simplista e pouco elucidativo quanto à disposição dos elementos de gestão, refletindo a incerteza inicial acerca da melhor forma de organizar visualmente as funcionalidades administrativas. A versão final, consequentemente, beneficiou de uma reformulação significativa, incorporando uma dashboard mais estruturada e completa, com indicadores e ferramentas de gestão que facilitam a tarefa de moderação e administração da plataforma.



Figura 24 - Página administrativa (Dashboard)

2.4. BASE DE DADOS

2.4.1. ENQUADRAMENTO TECNOLÓGICO

A base de dados do projeto foi gerida localmente através do ambiente de desenvolvimento Laragon, recorrendo ao sistema de gestão de bases de dados relacionais MySQL. As tabelas foram criadas através de migrações da *framework* Laravel, permitindo a definição e evolução controlada da estrutura da base de dados ao longo do desenvolvimento. Adicionalmente, foram implementados seeders para povoar algumas tabelas com dados iniciais, garantindo que o sistema dispõe de informação de base sem necessidade de intervenção manual.

A imagem do DER (Diagrama Entidade-Relacionamento) – serviu como guia visual para a implementação. Seguidamente, descrevem-se todas as entidades, os seus atributos, chaves e relacionamentos.

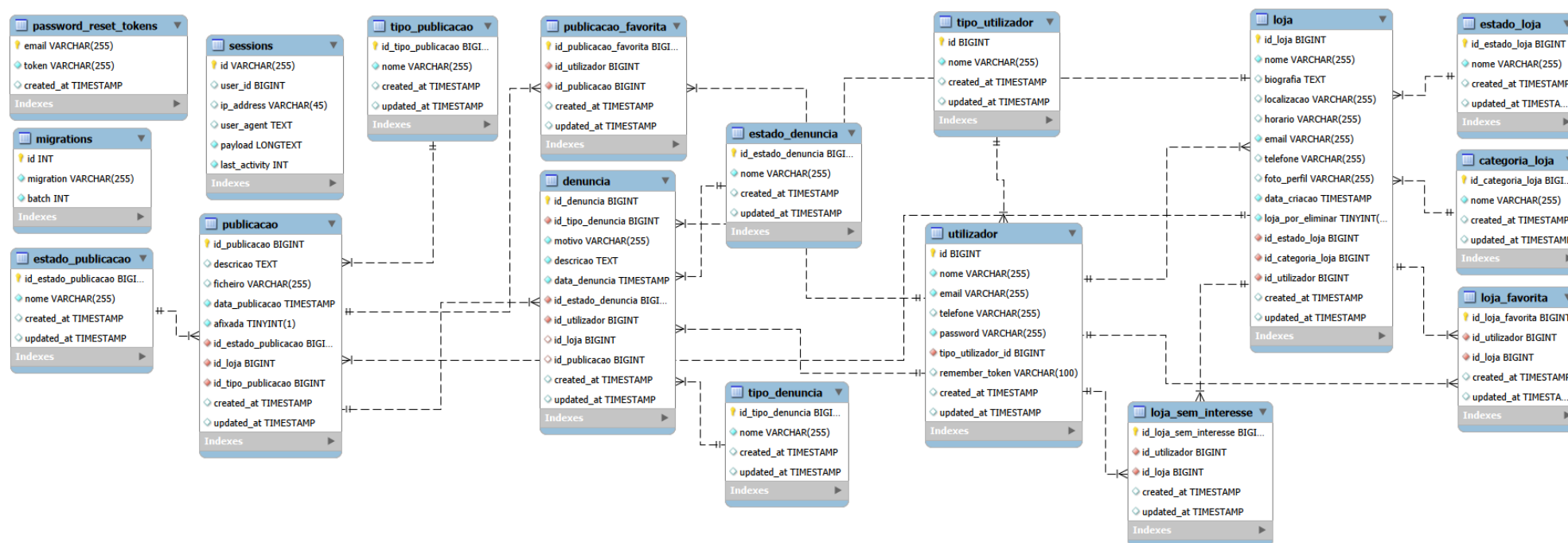


Figura 25 - Diagrama Entidade-Relação da base de dados do ComércioLocal

2.4.2. LISTAGEM DE ENTIDADES (TABELAS)

O modelo físico da base de dados contempla as seguintes tabelas, organizadas por domínio funcional:

- Domínio de utilizadores e autenticação: utilizador, tipo_utilizador, sessions, password_reset_tokens, migrations
- Domínio de lojas: loja, categoria_loja, estado_loja, loja_favorita, loja_sem_interesse
- Domínio de publicações: publicacao, tipo_publicacao, estado_publicacao, publicacao_favorita
- Domínio de denúncias: denuncia, tipo_denuncia, estado_denuncia

2.4.3. DESCRIÇÃO DE CADA TABELA

Segue-se uma descrição pormenorizada de cada uma das tabelas que compõem a base de dados, incluindo a sua finalidade, os campos mais relevantes e o papel que desempenham no funcionamento da plataforma.

Tabela utilizador

Esta tabela constitui o núcleo do sistema de gestão de utilizadores, armazenando todas as informações pessoais necessárias para a identificação e autenticação de cada utilizador da plataforma. Contempla campos como o nome completo e o número de telefone, para efeitos de identificação pessoal, bem como o endereço de correio eletrónico e a palavra-passe (devidamente hashed), utilizados no processo de início de sessão. Adicionalmente, inclui uma chave estrangeira para a tabela tipo_utilizador, que define o perfil de acesso de cada utilizador.

Tabela tipo_utilizador

Esta tabela desempenha um papel fundamental no controlo de acessos e na definição de permissões, uma vez que permite identificar o tipo de utilizador que está a interagir com a plataforma. Através do identificador armazenado na tabela utilizador, o backend consegue, em tempo de execução, bloquear o acesso a determinadas páginas ou, em alternativa, disponibilizar funcionalidades específicas consoante o perfil em causa. Esta tabela é pré-preenchida através de seeders, contendo três tipos de utilizador: visitante (utilizador comum autenticado), lojista (comerciante com painel de controlo) e administrador (gestor da plataforma com poderes de moderação).

Tabela sessions

Esta tabela é gerada automaticamente pelo framework Laravel e tem como finalidade armazenar as sessões dos utilizadores. Sempre que um utilizador inicia sessão, é criada uma nova

sessão que regista informações contextuais, designadamente o dispositivo a partir do qual o acesso é efetuado (computador ou telemóvel), o navegador utilizado (Chrome, Firefox, entre outros) e o endereço IP. Estes dados permitem ao sistema manter o estado de autenticação do utilizador ao longo da sua navegação e, simultaneamente, contribuem para a segurança da plataforma.

Tabela password_reset_tokens

Esta tabela é igualmente gerada pelo Laravel e destina-se a suportar o mecanismo de recuperação de palavra-passe. Quando um utilizador solicita a reposição da sua palavra-passe, é gerado um token único que é armazenado nesta tabela, associado ao endereço de correio eletrónico do utilizador. Este token é posteriormente enviado por correio eletrónico e permite ao utilizador redefinir a sua palavra-passe através de um link seguro, com validade temporal limitada, garantindo assim a segurança do processo.

Tabela migrations

Esta tabela é criada pelo Laravel e tem como função registar o histórico de migrações executadas na base de dados. Cada migração corresponde a uma alteração estrutural (criação, alteração ou eliminação de tabelas), e a tabela migrations armazena o nome da migração e a data/hora em que foi executada. Este mecanismo permite ao Laravel manter um controlo rigoroso sobre o estado da estrutura da base de dados, facilitando a aplicação de alterações de forma controlada e a reversão para estados anteriores, se necessário.

Tabela loja

Esta tabela é o elemento central do domínio comercial da plataforma, armazenando todas as informações necessárias para a identificação e caracterização de cada estabelecimento. Contempla campos como o nome da loja, o endereço de correio eletrónico e o número de telefone para contacto, a morada física (incluindo código postal), as coordenadas georreferenciadas (latitude e longitude) para efeitos de localização em mapas, o horário de funcionamento, uma descrição textual do estabelecimento e a URL da imagem de capa. Inclui, ainda, chaves estrangeiras para as tabelas estado_loja e tipo_utilizador (associando a loja ao utilizador lojista que a gere).

Tabela categoria_loja

Análoga à tabela tipo_utilizador, esta tabela permite classificar os estabelecimentos de acordo com a natureza da sua atividade comercial. Cada loja pode estar associada a uma ou mais categorias (por exemplo, Restauração, Ourivesaria, Talho, Papelaria, entre outras), possibilitando aos utilizadores pesquisar e filtrar lojas por tipo de negócio. Esta tabela é pré-preenchida através de seeders, garantindo a disponibilidade imediata de um conjunto de categorias predefinidas no momento da instalação da plataforma.

Tabela estado_loja

A tabela estado_loja define o estado operacional de cada estabelecimento na plataforma, permitindo ao administrador gerir a visibilidade e o acesso às lojas. Através de seeders, são pré-

definidos três estados possíveis: ativo, que indica que a loja se encontra visível e acessível a todos os utilizadores autenticados; bloqueado, que corresponde a uma suspensão temporária do estabelecimento, geralmente aplicada na sequência de uma infração às regras da plataforma; e pendente, que assinala que a loja foi registada pelo comerciante, mas aguarda a aprovação por parte do administrador para ser publicada.

Tabela loja_favorita

Esta tabela estabelece uma relação de preferência entre um utilizador do tipo visitante e uma determinada loja. Armazena apenas duas chaves estrangeiras: o identificador do utilizador e o identificador da loja. Esta estrutura permite que cada visitante possa guardar os seus estabelecimentos favoritos, facilitando o acesso rápido aos mesmos em futuras visitas à plataforma e melhorando a experiência de utilização.

Tabela loja_sem_interesse

Semelhante em estrutura à tabela loja_favorita, esta tabela tem como finalidade registar as lojas pelas quais um utilizador manifestou desinteresse. Ao guardar o identificador do utilizador e o identificador da loja, a plataforma pode, em futuras pesquisas e listagens, ocultar esses estabelecimentos, personalizando a experiência de navegação de acordo com as preferências expressas pelo utilizador.

Tabela publicacao

Esta tabela armazena todas as publicações efetuadas pelos comerciantes no âmbito dos seus estabelecimentos. Cada publicação é composta por uma descrição textual, de preenchimento obrigatório, e pode, opcionalmente, incluir uma fotografia ou um vídeo. A tabela contempla, ainda, um campo booleano que permite ao lojista "afixar" uma publicação, destacando-a no topo do perfil da sua loja. Inclui chaves estrangeiras para as tabelas loja (identificando o estabelecimento a que pertence), tipo_publicacao (classificando o conteúdo) e estado_publicacao (definindo a sua visibilidade).

Tabela tipo_publicacao

A tabela tipo_publicacao tem como finalidade classificar as publicações de acordo com o formato do conteúdo que veiculam, sendo igualmente pré-preenchida através de seeders com três tipos distintos: texto, para publicações compostas exclusivamente por conteúdo textual; fotografia, para publicações que incluem uma imagem como elemento principal; e vídeo, para publicações que incorporam um vídeo. Esta classificação permite à plataforma exibir os conteúdos de forma adequada e otimizada, bem como disponibilizar filtros de pesquisa por tipo de conteúdo.

Tabela estado_publicacao

A tabela estado_publicacao define o estado de visibilidade de cada publicação, permitindo ao administrador e ao comerciante gerir a sua disponibilidade na plataforma. São considerados três estados, pré-definidos através de seeders: ativo, que indica que a publicação se encontra visível no

perfil da loja; bloqueado, que assinala que a publicação foi desativada, por exemplo, na sequência de uma denúncia validada; e arquivado, que corresponde à remoção da publicação da visualização ativa, mantendo-se, no entanto, na base de dados para efeitos de histórico.

Tabela publicação_favorita

Semelhante à tabela loja_favorita, esta tabela regista as publicações que um utilizador visitante assinalou como favoritas. Armazena os identificadores do utilizador e da publicação, permitindo ao visitante criar uma lista personalizada de conteúdos de interesse e aceder rapidamente aos mesmos.

Tabela denuncia

Esta tabela regista as denúncias submetidas pelos utilizadores visitantes relativamente a conteúdos ou estabelecimentos que considerem inadequados ou violadores das regras da plataforma. Compreende dois campos de preenchimento obrigatório por parte do utilizador: o motivo da denúncia (selecionado a partir de uma lista pré-definida) e uma descrição detalhada da situação. A tabela inclui, ainda, chaves estrangeiras para as tabelas tipo_denuncia (classificando o alvo da denúncia), estado_denuncia (definindo o andamento do processo) e utilizador (identificando o autor da denúncia), bem como campos opcionais para associar a denúncia a uma loja ou a uma publicação específica.

Tabela tipo_denuncia

A tabela tipo_denuncia, define os dois tipos possíveis de denúncia que podem ser submetidos na plataforma, sendo pré-preenchida através de seeders com as seguintes opções: loja, quando a denúncia incide sobre um estabelecimento comercial (por exemplo, por informação incorreta ou conteúdo ofensivo); e publicação, quando a denúncia recai sobre uma publicação específica (por exemplo, por conteúdo inadequado ou spam). A definição clara do tipo de denúncia permite ao administrador encaminhar corretamente o processo de moderação e adotar as medidas adequadas.

Tabela estado_denuncia

Por fim, a tabela estado_denuncia, define o estado do processo de cada denúncia, permitindo ao administrador gerir o seu fluxo de trabalho. São considerados dois estados, igualmente pré-preenchidos através de seeders: pendente, que indica que a denúncia foi submetida, mas aguarda análise e deliberação por parte do administrador; e ação tomada, que assinala que a denúncia foi analisada e que foi adotada uma medida, como o bloqueio da loja, a remoção da publicação ou o arquivamento da denúncia, conforme o caso. Este mecanismo assegura que todas as denúncias são devidamente processadas, garantindo a integridade e a qualidade da plataforma.

2.4.4. RELACIONAMENTOS ENTRE TABELAS (CHAVES ESTRANGEIRAS)

Os relacionamentos entre as entidades seguem uma lógica de integridade referencial baseada em chaves estrangeiras, garantindo a coerência dos dados. Um utilizador encontra-se associado a um tipo de utilizador numa relação de muitos para um (N:1), sendo que cada tipo pode abranger

vários utilizadores. Cada loja está associada a um utilizador, numa relação de um para um (1:1). As lojas estão associadas às categorias e estados, mantendo também relações de cardinalidade N:1 com estas entidades.

No caso das publicações, cada uma está associada a uma loja, a um tipo de publicação e a um estado, seguindo igualmente relações de muitos para um.

Existem ainda relações de interação entre utilizadores e conteúdos, nomeadamente lojas favoritas, publicações favoritas e lojas marcadas como “sem interesse”, todas implementadas através de tabelas de associação que permitem relações muitos-para-muitos.

Por fim, as denúncias permitem que os utilizadores reportem lojas ou publicações, estando associadas a um tipo e estado de denúncia, garantindo a correta gestão e tratamento dos casos reportados.

2.4.5. INTEGRIDADE REFERENCIAL E AÇÕES EM CASCATA

Todas as chaves estrangeiras foram implementadas com restrição ON DELETE RESTRICT ou ON DELETE CASCADE, conforme apropriado:

publicacao → loja: ON DELETE CASCADE

Caso uma loja seja eliminada, todas as suas publicações associadas são automaticamente removidas.

loja_favorita / loja_sem_interesse → loja e utilizador: ON DELETE CASCADE

Se um utilizador ou uma loja for eliminado, os respetivos registos de favoritos ou de “não interesse” são automaticamente apagados.

denuncia → utilizador: ON DELETE CASCADE

Se o utilizador que realizou a denúncia for eliminado, as denúncias associadas também são removidas.

denuncia → loja / publicacao: ON DELETE SET NULL

Caso a loja ou publicação denunciada seja eliminada, a denúncia é mantida para fins de histórico, sendo apenas removida a referência à entidade em causa.

2.4.6. POVOAMENTO INICIAL (SEEDERS)

Os seeders são componentes do Laravel que têm como finalidade preencher a base de dados com dados iniciais de referência, de forma automatizada e controlada. Estes dados, também designados por dados mestres ou de configuração, são essenciais para o funcionamento da aplicação,

uma vez que definem as opções disponíveis em menus dropdown, os estados possíveis de registos e os perfis de utilizador, entre outros.

No contexto do desenvolvimento de software, os seeders desempenham um papel fundamental por várias razões. Em primeiro lugar, eliminam a necessidade de inserção manual de dados, um processo moroso e sujeito a erros, especialmente quando existem muitos registos a criar. Em segundo lugar, garantem a consistência dos dados entre diferentes ambientes de desenvolvimento, teste e produção, assegurando que todos os programadores e testadores trabalham com o mesmo conjunto de dados de base. Em terceiro lugar, facilitam a execução de testes automatizados, uma vez que os seeders podem ser executados antes de cada bateria de testes para garantir que a base de dados se encontra num estado conhecido e previsível. Por último, os seeders permitem recriar rapidamente a estrutura de dados inicial sempre que necessário, sem depender de configurações manuais.

Para o projeto ComércioLocal, foram desenvolvidos seeders específicos para popular a base de dados com os seguintes dados de referência, organizados por domínio funcional:

- `tipo_utilizador` – define os três perfis de acesso à plataforma: Administrador, Lojista e Visitante, permitindo ao sistema distinguir as permissões e funcionalidades disponíveis para cada tipo de utilizador.
- `categoria_loja` – contém um conjunto de 79 categorias pré-definidas, abrangendo os mais diversos setores de atividade comercial, como Restauração, Ourivesaria, Talho, Papelaria, entre outras. Esta diversidade permite que os lojistas classifiquem corretamente os seus estabelecimentos e que os visitantes possam pesquisar e filtrar lojas por categoria de forma eficiente.
- `estado_loja` – define os três estados possíveis para uma loja: Ativa, Bloqueada e Pendente. Estes estados permitem ao administrador gerir o ciclo de vida dos estabelecimentos, desde o registo inicial até à aprovação ou bloqueio.
- `tipo_publicacao` – classifica as publicações dos lojistas em Imagem, Vídeo e Texto, permitindo à plataforma adaptar a apresentação dos conteúdos consoante o seu formato.
- `estado_publicacao` – define os estados de visibilidade das publicações: Ativa (visível no perfil da loja), Bloqueada (desativada por motivos de moderação) e Arquivada (removida da visualização ativa, mas mantida na base de dados para histórico).
- `tipo_denuncia` – estabelece os dois tipos de denúncia que os visitantes podem submeter: Loja (incidindo sobre um estabelecimento) e Publicação (incidindo sobre um conteúdo específico).
- `estado_denuncia` – define os estados de processamento das denúncias: Pendente (aguarda análise por parte do administrador) e Resolvida (já foi analisada e foi tomada uma decisão).

A utilização de seeders no projeto ComércioLocal revelou-se particularmente vantajosa, uma vez que garantiu que a plataforma pudesse ser testada e utilizada imediatamente após a instalação, sem necessidade de configuração manual inicial. Ao executar os seeders, o sistema fica imediatamente operacional, com todos os dados de referência disponíveis, o que acelera o ciclo de desenvolvimento e facilita a demonstração do projeto a terceiros.

3. IMPLEMENTAÇÃO DO PROJETO

O presente capítulo tem como objetivo descrever, de forma detalhada e sequencial, as etapas mais importantes do processo de desenvolvimento do projeto ComércioLocal, desde a configuração inicial do ambiente de desenvolvimento até à implementação das funcionalidades que compõem a plataforma. A descrição aqui apresentada visa documentar o percurso técnico percorrido, permitindo uma compreensão clara das decisões adotadas e dos procedimentos executados ao longo de toda a fase de implementação.

3.1. CONFIGURAÇÃO DO AMBIENTE E CRIAÇÃO DO PROJETO

A primeira etapa do desenvolvimento consistiu na configuração do ambiente de trabalho, que incluiu a instalação e configuração de todas as ferramentas e tecnologias necessárias para suportar o desenvolvimento da aplicação web. Este processo foi executado num sistema operativo Windows, tendo sido adotado um conjunto de ferramentas integradas que permitem uma gestão eficiente do ciclo de desenvolvimento.

3.1.1. INSTALAÇÃO DO LARAGON

O primeiro passo consistiu na instalação do Laragon, um ambiente de desenvolvimento local que integra, de forma simplificada, um servidor web, um sistema de gestão de bases de dados e outras ferramentas essenciais para o desenvolvimento em PHP. A versão utilizada no projeto foi a Laragon 2026 v8.6.1 260302 [default], que oferece uma interface intuitiva e permite um controlo centralizado sobre os serviços em execução.

3.1.2. INSTALAÇÃO E CONFIGURAÇÃO DO PHP

De seguida, procedeu-se à instalação do PHP, a linguagem de programação que serve de base ao framework Laravel. A versão utilizada foi a php-8.5.4-Win32-vs17-x64, cujo ficheiro, disponibilizado no formato .zip, foi descarregado e extraído. O conteúdo extraído, correspondente a uma pasta com o nome da versão, foi movido para o diretório de instalação do Laragon, no seguinte caminho: C:\laragon\bin\php. Conforme ilustrado na Figura 25, a pasta php-8.5.4-Win32-vs17-x64 foi colocada neste local, juntamente com as outras versões de PHP eventualmente existentes.

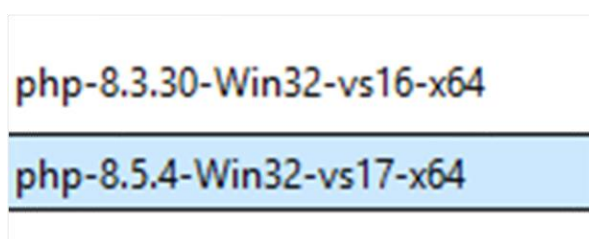


Figura 26 – Pasta php-8.5.4-Win32-vs17-x64 no diretório do Laragon

Após a colocação da pasta no local adequado, o Laragon foi reiniciado e, através da sua interface, foi selecionada a nova versão do PHP como versão ativa, garantindo que todas as operações subsequentes utilizassem esta configuração, conforme se observa na Figura 26.

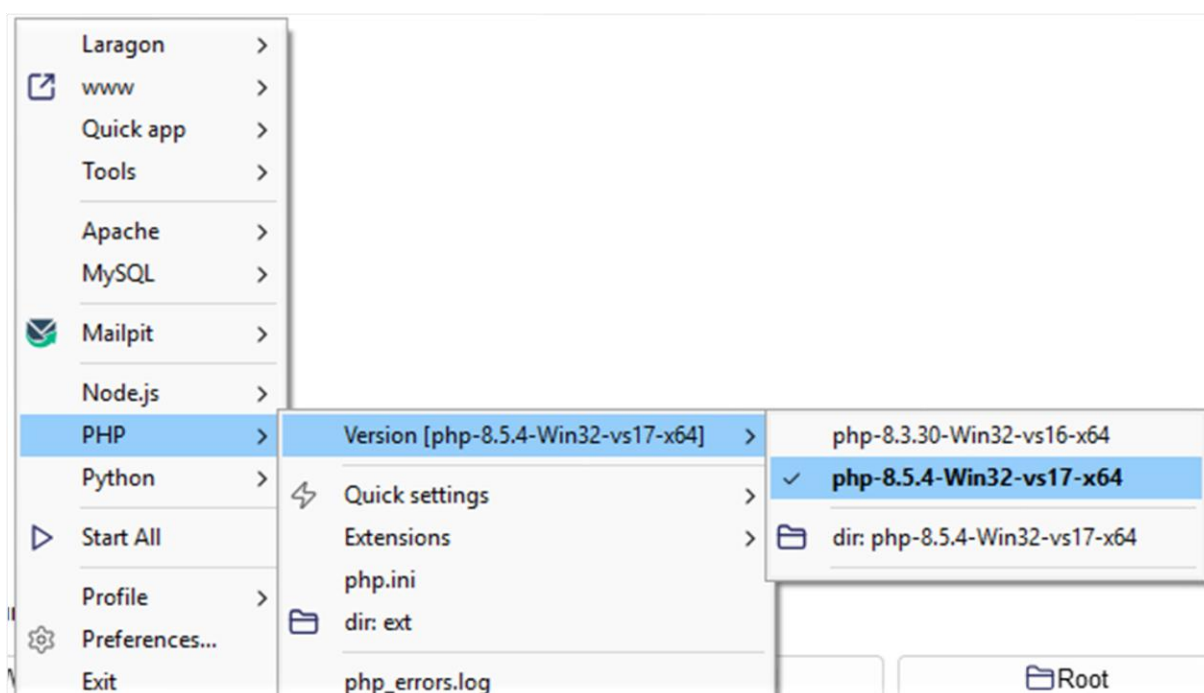


Figura 27 – Definição da nova versão instalada do PHP no Laragon

De seguida, procedeu-se à configuração do ficheiro `php.ini`, que contém as definições de configuração do PHP. Para ativar a extensão `zip`, que é necessária para a manipulação de ficheiros comprimidos, abriu-se o referido ficheiro através do menu do Laragon (Figura 27). Dentro do ficheiro, localizou-se a linha 813, que continha o texto `;zip` (com o ponto e vírgula a indicar que a extensão se encontrava comentada e, portanto, desativada). Removeu-se o ponto e vírgula e guardou-se o ficheiro, conforme evidencia a Figura 28, ativando assim a extensão.

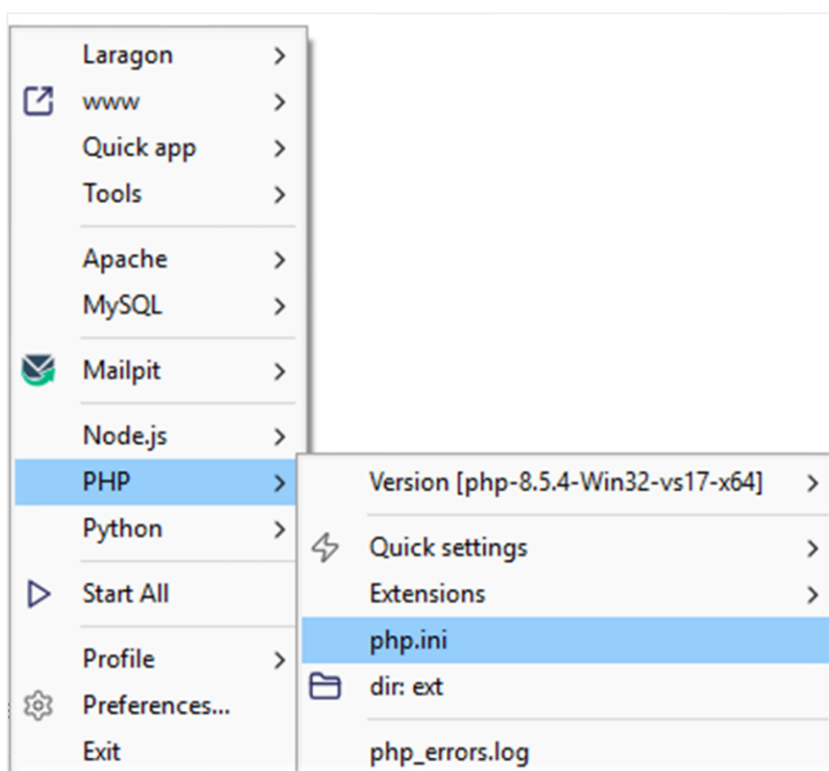


Figura 28 – Abertura do ficheiro php.ini através do Laragon

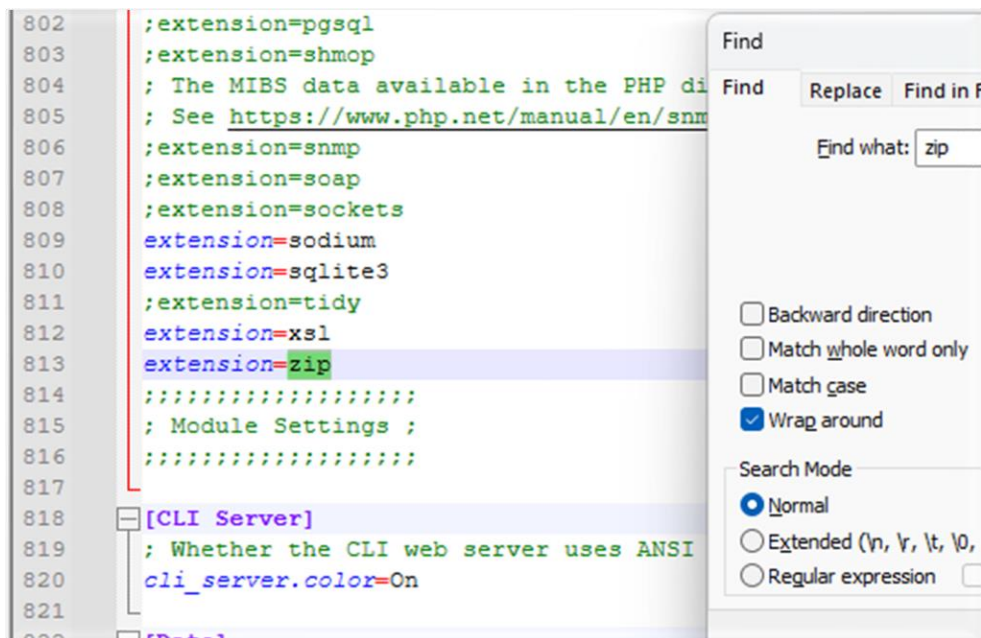


Figura 29 – Localização da palavra "zip" e remoção do ";" na linha 813

3.1.3. INSTALAÇÃO DO COMPOSER

O Composer é um gestor de dependências para PHP, essencial para a instalação do Laravel e de todos os pacotes necessários ao projeto. A versão utilizada foi a Composer 2.10.1. Durante o processo de instalação, foi necessário indicar a localização do executável do PHP. Para o efeito, navegou-se até ao diretório C:\laragon\bin\php\php-8.5.4-Win32-vs17-x64 e selecionou-se o ficheiro php.exe, garantindo que o Composer ficasse devidamente associado à versão de PHP previamente configurada.

3.1.4. INSTALAÇÃO DO NODE.JS

O Node.js é uma plataforma que permite a execução de JavaScript no lado do servidor e é utilizado, no contexto deste projeto, para a gestão dos pacotes e dependências do frontend, incluindo a compilação de recursos como CSS e JavaScript. A versão instalada foi a Node 24.14.1, que assegura a compatibilidade com as bibliotecas e ferramentas utilizadas.

3.1.5. CRIAÇÃO DO REPOSITÓRIO NO GITHUB

Para garantir a gestão de versões do código-fonte e a segurança do trabalho desenvolvido, foi criado um repositório privado no GitHub, com a designação "Repositorio_ComercioLocal" (Figura 29). Esta opção permite armazenar o código na cloud, facilitar o controlo de alterações e assegurar a recuperação do trabalho em caso de falhas no computador local.

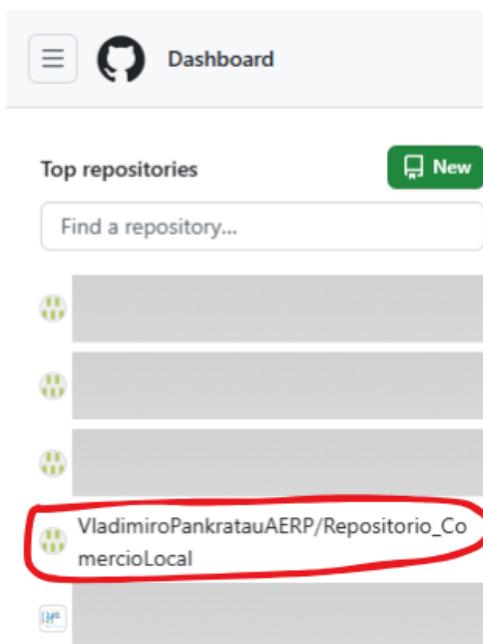


Figura 30 - Repositório ComercioLocal no GitHub

3.1.6. CLONAGEM DO REPOSITÓRIO

No computador local, foi criada uma pasta com o nome "PAP-Vladimiro_Pankratau", destinada a albergar todos os ficheiros do projeto. Através da aplicação GitKraken, que oferece uma interface gráfica para o Git, foi efetuado o clone do repositório criado no GitHub para a pasta recém-criada (Figura 30). Este procedimento descarregou todos os ficheiros do repositório para o computador local, estabelecendo a ligação entre o ambiente local e o repositório remoto.

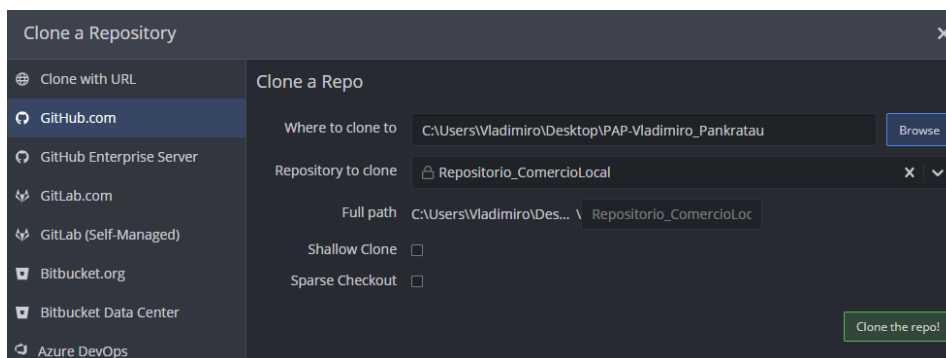


Figura 31 - Clone do repositório no GitKraken

3.1.7. CRIAÇÃO DO PROJETO LARAVEL

Com o ambiente de desenvolvimento configurado, procedeu-se à abertura do Visual Studio Code, o editor de código utilizado ao longo do projeto, e à abertura da pasta PAP-Vladimiro_Pankratau/Repositorio_ComercioLocal. Dentro do terminal integrado do Visual Studio Code, executou-se o comando `composer global require laravel/installer` (Figura 31), que instala globalmente o instalador do Laravel.

```
composer global require laravel/installer
```

Figura 32 - Comando `composer global require laravel/installer`

De seguida, ainda no terminal, executou-se o comando `laravel new ComercioLocal` (Figura 32), que permite criar um novo projeto Laravel com o nome "ComercioLocal". Durante a execução deste comando, foram apresentadas algumas perguntas de configuração, às quais se respondeu da seguinte forma: primeiro "none", segundo "0" e por último "no".

```
laravel new ComercioLocal
```

Figura 33 - Comando `laravel new ComercioLocal`

```

Which starter kit would you like to install? [None]:
[none  ] None
[react ] React
[vue   ] Vue
[livewire] Livewire
> none

Which testing framework do you prefer? [Pest]:
[0] Pest
[1] PHPUnit
> 0

Do you want to install Laravel Boost to improve AI assisted coding? [yes]:
> no

```

Figura 34 - Comando laravel new ComercioLocal

Posteriormente, foram apresentadas duas perguntas adicionais (Figura 33), relativas à configuração da base de dados, tendo-se respondido mysql (para utilizar o MySQL como sistema de gestão de base de dados) e no (para não executar outras configurações adicionais naquele momento).

```

Which database will your application use? [SQLite]:
[sqlite] | SQLite
[mysql] | MySQL
[mariadb] | MariaDB
[pgsql] | PostgreSQL (Missing PDO extension)
[sqlsrv] | SQL Server (Missing PDO extension)
> mysql

Default database updated. Would you like to run the default
database migrations? (yes/no) [yes]:
> no

```

Figura 35 - Configuração do laravel new ComercioLocal

De seguida, executaram-se, de forma consecutiva, os comandos `cd ComercioLocal` (para navegar para o diretório do projeto), `npm install` (para instalar as dependências do frontend) e `npm run build` (para compilar os assets do projeto), conforme ilustrado nas Figuras 34 e 35.

```
cd ComercioLocal
npm install
```

Figura 36 - Comandos cd ComercioLocal e npm install

```
npm run build
```

Figura 37 - Comando npm run build

3.1.8. CONFIGURAÇÃO DA BASE DE DADOS

Após a criação do projeto, procedeu-se à configuração da base de dados. No Laragon, iniciou-se o serviço MySQL 8.4.3 (Figura 36), que disponibiliza o sistema de gestão de bases de dados relacional.

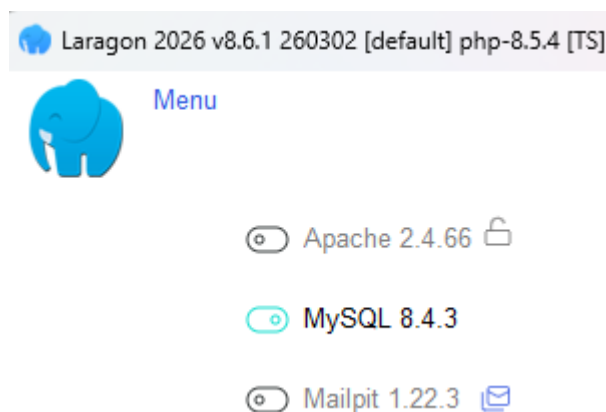


Figura 38 - Inicialização do MySQL no Laragon

De seguida, abriu-se o MySQL Workbench, uma ferramenta visual para gestão de bases de dados, e criou-se uma base de dados com o nome comerciocal (Figura 37).

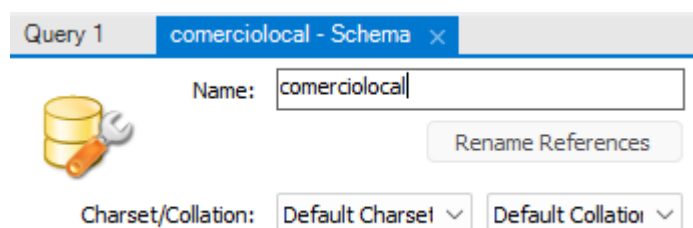


Figura 39 - Criação da base de dados comerciocal no MySQL Workbench

No Visual Studio Code, dentro da pasta do projeto ComercioLocal, localizou-se o ficheiro .env, que contém as variáveis de ambiente do Laravel. Neste ficheiro, foram configuradas as definições de ligação à base de dados, preenchendo os campos com os dados correspondentes à configuração do

MySQL Workbench (nome da base de dados, utilizador, palavra-passe, host e porta), conforme se pode observar na Figura 38.

```

23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=comerciocal
27 DB_USERNAME=root
28 DB_PASSWORD=root

```

Figura 40 - Configuração da ligação à base de dados comerciocal no ficheiro .env

3.1.9. CONFIGURAÇÃO DO MAILTRAP

Para suportar o envio de correio eletrónico no ambiente de desenvolvimento, nomeadamente para funcionalidades como a recuperação de palavra-passe e a notificação de registos, foi utilizada a ferramenta Mailtrap. O Mailtrap é um serviço online que simula um servidor de correio eletrónico, permitindo testar o envio de mensagens sem as enviar efetivamente para destinatários reais, o que é particularmente útil durante a fase de desenvolvimento.

O primeiro passo consistiu em aceder ao site oficial do Mailtrap (mailtrap.io) e criar uma sandbox na aba "Sandboxes", à qual foi atribuído o nome comerciocal, conforme ilustrado na Figura 39. Uma sandbox funciona como um ambiente de teste isolado, onde todas as mensagens enviadas são capturadas e disponibilizadas para consulta, sem serem encaminhadas para os destinatários finais.

Projects

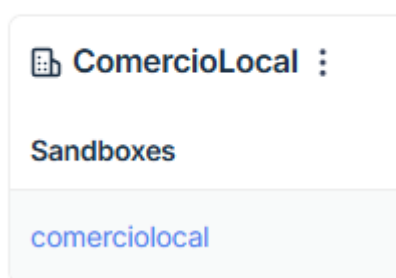


Figura 41 - Sandbox comerciocal no Mailtrap

De seguida, acedeu-se à secção "Settings" da sandbox comerciocal e navegou-se até à opção "Code Samples". Neste local, selecionou-se a opção "PHP: Laravel 9+", que disponibiliza o código de configuração específico para integração com o *framework* Laravel, conforme se observa na Figura 40.

Code Samples

cURL **PHP: Laravel 9+** Node.js Python C# Ruby Other

```

1  MAIL_MAILER=smtp
2  MAIL_HOST=sandbox.smtp.mailtrap.io
3  MAIL_PORT=2525
4  MAIL_USERNAME=5c5fc32b1adf23
5  MAIL_PASSWORD=****2402

```

Figura 42 - Código de configuração do Mailtrap para Laravel 9+

O código disponibilizado, composto por cinco linhas de configuração, foi copiado para a área de transferência. De seguida, abriu-se o ficheiro .env do projeto ComercioLocal no Visual Studio Code e localizaram-se as linhas correspondentes às variáveis de configuração do correio eletrónico (MAIL_*), como se pode verificar na Figura 41.

```

50  MAIL_MAILER=smtp
51  MAIL_SCHEME=null
52  MAIL_HOST=sandbox.smtp.mailtrap.io
53  MAIL_PORT=2525
54  MAIL_USERNAME=bf1a80350bd676
55  MAIL_PASSWORD=d8149c42e43773
56  MAIL_FROM_ADDRESS="comerciocal@comercio.pt"
57  MAIL_FROM_NAME="${APP_NAME}"

```

Figura 43 - Configuração do Mailtrap no ficheiro .env

Procedeu-se, então, à substituição das variáveis existentes no ficheiro .env pelos valores fornecidos pelo Code Samples do Mailtrap, nomeadamente as variáveis MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME e MAIL_PASSWORD, que foram preenchidas com os respetivos valores disponibilizados pela plataforma. Adicionalmente, a variável MAIL_FROM_ADDRESS foi preenchida com um endereço de correio eletrónico fictício, que será utilizado como remetente das mensagens simuladas no projeto. Este endereço pode ser qualquer um, uma vez que as mensagens não são efetivamente enviadas para destinatários reais, servindo apenas para identificar o remetente nas mensagens capturadas e disponibilizadas para consulta no Mailtrap. Desta forma, o Laravel fica configurado para utilizar o Mailtrap como driver de correio eletrónico, permitindo testar todas as funcionalidades que envolvem o envio de mensagens, como a recuperação de palavra-passe e a notificação de registos, sem necessidade de configurar um servidor de correio real.

3.1.10. INICIALIZAÇÃO DO SERVIDOR

Por fim, de volta ao terminal do Visual Studio Code, executou-se o comando `php artisan serve` (Figura 42), que inicia o servidor de desenvolvimento integrado do Laravel, disponibilizando a aplicação num endereço local.



Figura 44 - Inicialização do servidor com `php artisan serve`

No navegador, acedeu-se ao endereço `http://localhost:8000`, tendo sido apresentada a página inicial padrão do Laravel (Figura 43), confirmando que a instalação e configuração do ambiente foram realizadas com sucesso e que o projeto se encontrava pronto para dar início à fase de desenvolvimento das funcionalidades específicas do ComércioLocal.

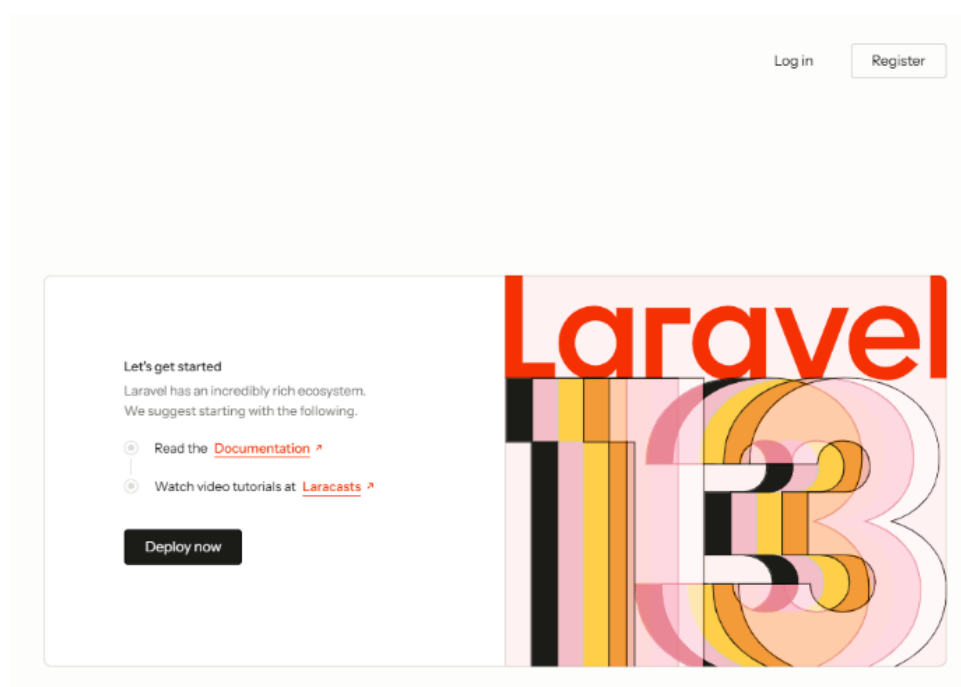


Figura 45 - Página inicial padrão do Laravel

3.2. ESTRUTURA DE PASTAS DO LARAVEL

O framework Laravel adota uma estrutura de diretórios padronizada e bem organizada, que facilita o desenvolvimento, a manutenção e a escalabilidade da aplicação. Cada pasta e ficheiro desempenha um papel específico no funcionamento do projeto, sendo importante compreender a sua função para uma correta gestão do código. De seguida, apresenta-se uma descrição detalhada de cada um dos diretórios e ficheiros que compõem a estrutura do projeto ComércioLocal.

3.2.1. PASTA APP

A pasta app é o coração da aplicação, contendo a lógica central do negócio (business logic). É aqui que se encontram os modelos, que interagem com a base de dados, os controladores, que processam os pedidos dos utilizadores e devolvem as respostas adequadas, os provedores de serviço, que registam e configuram serviços do Laravel, e outras classes fundamentais, como middlewares, requests e policies. Esta pasta é organizada em subdiretórios consoante a funcionalidade, sendo que os mais relevantes para o projeto são a subpasta Http/Controllers, que contém os controladores que recebem os pedidos HTTP, executam a lógica necessária e retornam as views ou respostas JSON, a subpasta Http/Middleware, que armazena os middlewares que atuam como filtros nos pedidos HTTP, validando, por exemplo, se o utilizador está autenticado, a subpasta Models, que contém as classes que representam as tabelas da base de dados, permitindo a interação com a mesma através do Eloquent ORM, e a subpasta Providers, que contém os provedores de serviço, que registam serviços no contentor do Laravel e configuram a aplicação no arranque. A Figura 44 ilustra o conteúdo da pasta app do projeto.

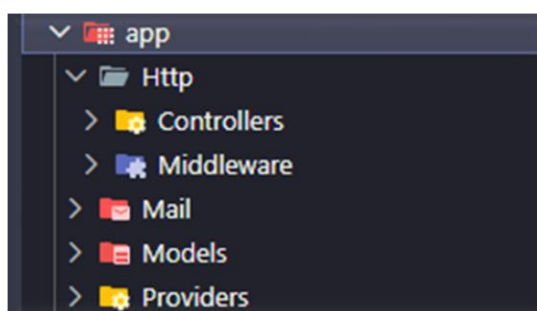


Figura 46 - Conteúdo da pasta app

3.2.2. PASTA BOOTSTRAP

A pasta bootstrap contém os ficheiros responsáveis pelo arranque (bootstrap) do framework Laravel, incluindo o ficheiro app.php, que inicia a aplicação e carrega todos os serviços essenciais, e a subpasta cache, onde são armazenados os ficheiros de configuração e de rotas em cache, para otimização do desempenho.

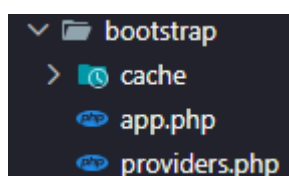


Figura 47 - Conteúdo da pasta bootstrap

3.2.3. PASTA CONFIG

A pasta config alberga todos os ficheiros de configuração da aplicação, cada um controlando um aspeto específico, como a configuração da base de dados no ficheiro database.php, do correio eletrónico no ficheiro mail.php, da autenticação no ficheiro auth.php, dos serviços de ficheiros no ficheiro filesystems.php, entre outros. Estes ficheiros são carregados durante o arranque da aplicação e podem ser acedidos através do helper config().

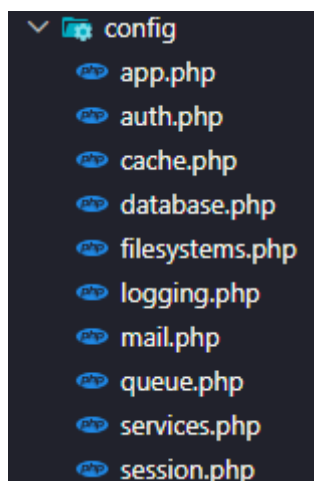


Figura 48 - Conteúdo da pasta config

3.2.4. PASTA DATABASE

A pasta database contém todos os ficheiros relacionados com a base de dados e a gestão de dados da aplicação, sendo composta pelas subpastas migrations, que contém os ficheiros de migração que definem a estrutura das tabelas da base de dados de forma versionada, permitindo criar, modificar ou reverter a estrutura da base de dados de forma controlada, pela subpasta seeders, que contém os ficheiros de seeders utilizados para povoar a base de dados com dados iniciais de teste ou de desenvolvimento, como categorias, tipos de utilizador e estados, e pela subpasta factories, que contém as fábricas que permitem gerar grandes quantidades de dados falsos para testes. A Figura 47 ilustra o conteúdo da pasta database do projeto.

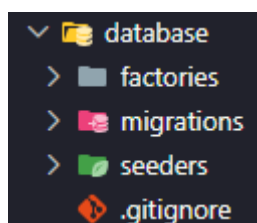


Figura 49 - Conteúdo da pasta database

3.2.5. PASTA PUBLIC

A pasta public é o diretório raiz do servidor web, sendo o ponto de entrada da aplicação, onde todos os ficheiros colocados são acessíveis publicamente, incluindo o ficheiro index.php, que processa todos os pedidos e os encaminha para a aplicação Laravel. Além disso, esta pasta contém recursos estáticos como imagens, CSS, JavaScript e ficheiros de favicon. A subpasta storage, que se encontra dentro de public, serve como ponto de ligação simbólica para os ficheiros carregados pelos utilizadores, que são armazenados em storage/app/public. A Figura 48 apresenta o conteúdo da pasta public, enquanto a Figura 49 detalha o seu conteúdo com a ligação simbólica para a subpasta storage.

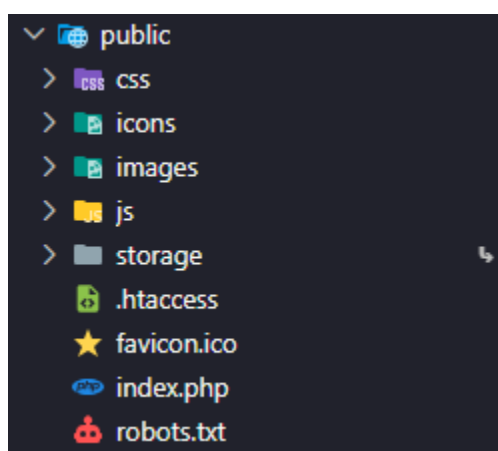


Figura 50 - Conteúdo da pasta public

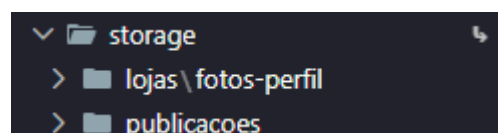


Figura 51 - Conteúdo da pasta public/storage

3.2.6. PASTA RESOURCES

A pasta resources contém os recursos da aplicação que não são compilados ou gerados automaticamente, incluindo as views (templates HTML), os ficheiros CSS e JavaScript, que são depois compilados, e os ficheiros de linguagem para internacionalização. A subpasta views é particularmente relevante, uma vez que contém as views da aplicação, escritas em Blade (o motor de templates do Laravel), que definem a camada de apresentação da plataforma, sendo aqui construídas todas as páginas visíveis ao utilizador, como a landing page, a página de login, o painel do comerciante e a dashboard do administrador. As subpastas css e js contêm os ficheiros de estilos e scripts que são compilados e otimizados para produção. A Figura 50 apresenta o conteúdo da pasta resources, enquanto a Figura 51 ilustra o conteúdo da subpasta views.

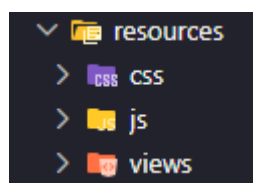


Figura 52 - Conteúdo da pasta resources

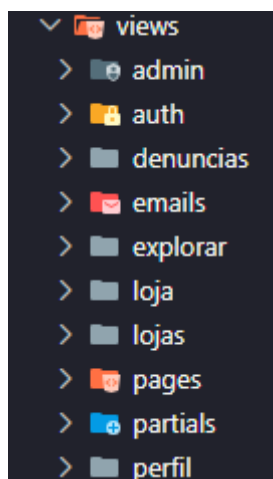


Figura 53 - Conteúdo da pasta resources/views

3.2.7. PASTA ROUTES

A pasta routes contém todos os ficheiros de definição de rotas da aplicação, cada um definindo um conjunto de rotas para um determinado propósito. O ficheiro web.php define as rotas para a interface web, ou seja, as páginas acessíveis através do navegador, enquanto o ficheiro api.php define as rotas para a API da aplicação, utilizadas para comunicação com o frontend ou com outras aplicações. O ficheiro console.php define os comandos Artisan personalizados e o ficheiro channels.php define os canais de transmissão para eventos em tempo real. A Figura 52 ilustra o conteúdo da pasta routes do projeto.

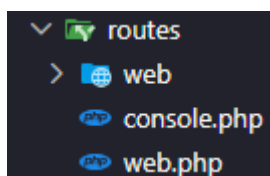


Figura 54 - Conteúdo da pasta routes

3.2.8. PASTA STORAGE

A pasta storage é utilizada para armazenar ficheiros gerados pela aplicação, incluindo ficheiros de sessão, ficheiros de cache, logs de sistema e ficheiros carregados pelos utilizadores, como imagens de lojas e publicações. A subpasta app/public é onde são armazenados os ficheiros carregados pelos

utilizadores, que são posteriormente acessíveis publicamente através de uma ligação simbólica criada a partir da pasta public/storage. A subpasta framework contém ficheiros de cache, sessões e views compiladas, enquanto a subpasta logs contém os ficheiros de registo (logs) da aplicação. A Figura 53 ilustra o conteúdo da subpasta storage/app/public, enquanto a Figura 54 apresenta o conteúdo da pasta storage do projeto.

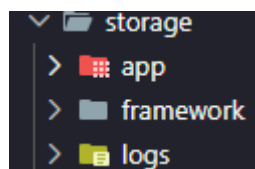


Figura 55 - Conteúdo da pasta storage

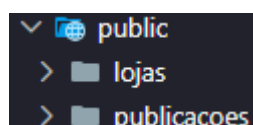


Figura 56 - Conteúdo da pasta storage/app/public

3.2.9. PASTA TESTS

A pasta tests contém os testes automatizados da aplicação, que permitem verificar o correto funcionamento das funcionalidades implementadas, sendo o Laravel compatível com testes unitários e de integração através do PHPUnit. Esta pasta é organizada em subpastas como Feature, para testes de funcionalidade, e Unit, para testes unitários.

3.2.10. PASTA VENDOR

A pasta vendor contém todas as dependências do projeto, ou seja, os pacotes e bibliotecas de terceiros instalados através do Composer, sendo gerida automaticamente por esta ferramenta e não devendo ser alterada manualmente. Inclui o próprio Laravel e todas as suas dependências, bem como outros pacotes adicionais que tenham sido instalados ao longo do desenvolvimento.

3.2.11. FICHEIROS NA RAIZ DO PROJETO

Na raiz do projeto, encontram-se vários ficheiros de configuração e gestão. O ficheiro .env contém as variáveis de ambiente, onde são definidas configurações sensíveis como a ligação à base de dados, as credenciais do Mailtrap, a chave da aplicação, entre outras, não devendo este ficheiro ser versionado no repositório, razão pela qual existe o ficheiro .env.example como modelo. O ficheiro .editorconfig define regras de formatação de código para editores de texto, garantindo a consistência do código entre diferentes programadores. Os ficheiros .gitattributes e .gitignore são ficheiros de configuração do Git, onde o .gitignore especifica quais ficheiros e pastas devem ser ignorados pelo

controlo de versões, como `vendor`, `node_modules` e `.env`, enquanto o `.gitattributes` define atributos para ficheiros, como a normalização de finais de linha. O ficheiro `artisan` é o executável da interface de linha de comandos do Laravel, que permite executar comandos personalizados, como criar controladores, modelos, migrações e seeders. Os ficheiros `composer.json` e `composer.lock` são ficheiros de configuração do Composer, onde o primeiro define as dependências do projeto e as suas versões, enquanto o segundo regista as versões exatas instaladas, garantindo a consistência entre diferentes ambientes. Os ficheiros `package.json` e `package-lock.json` são análogos, mas para as dependências do frontend (JavaScript, CSS, etc.). O ficheiro `phpunit.xml` é o ficheiro de configuração para a execução de testes com PHPUnit. O ficheiro `README.md` contém a documentação do projeto, com instruções de instalação e configuração. Por fim, o ficheiro `vite.config.js` é o ficheiro de configuração do Vite, que é a ferramenta de compilação de assets utilizada pelo Laravel para processar CSS e JavaScript. A Figura 55 apresenta a lista completa dos ficheiros e pastas na raiz do projeto.

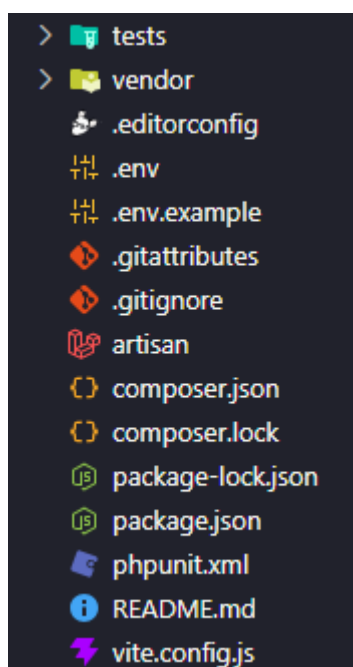


Figura 57 - Ficheiros e pastas na raiz do projeto

3.3. IMPLEMENTAÇÃO DA AUTENTICAÇÃO

3.3.1. CRIAÇÃO DAS ROTAS DE AUTENTICAÇÃO

A autenticação do sistema foi organizada através de rotas específicas para cada ação essencial, como iniciar sessão, criar conta, recuperar palavra-passe e terminar sessão. Esta separação permitiu manter o fluxo de autenticação claro e fácil de manter, garantindo que cada operação tivesse o seu próprio ponto de entrada na aplicação.

```

Route::get('/login', [AuthController::class, 'loginView'])->name('login');
Route::post('/login', [AuthController::class, 'login']);
Route::get('/forgot-password', [PasswordResetController::class, 'forgotPasswordView'])->name('password.request');
Route::post('/forgot-password', [PasswordResetController::class, 'sendResetLink'])->name('password.email');
Route::get('/reset-password/{token?}', [PasswordResetController::class, 'resetPasswordView'])->name('password.reset');
Route::post('/reset-password', [PasswordResetController::class, 'resetPassword'])->name('password.update');
Route::get('/register', [AuthController::class, 'registerView'])->name('register');
Route::post('/register', [AuthController::class, 'register']);
Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

```

Figura 58 - código das rotas de autenticação

3.3.2. PÁGINA DE LOGIN

Foi desenvolvida a interface de login onde o utilizador introduz o seu email e palavra-passe para aceder à plataforma. Nesta página também foi incluída a apresentação de mensagens de erro, para que o utilizador perceba imediatamente quando os dados inseridos estão incorretos ou quando existe algum problema com a conta.

Figura 59 - página com o formulário de login

```

<div class="form-group">
  <label for="loginEmail">Email</label>
  <input type="email" id="loginEmail" name="email" class="form-input" placeholder="Digite o seu email"
    value="{{ old('email') }}" required>
  @error('email')
  <span class="error-message" style="display: block;">{{ $message }}</span>
  @enderror
</div>

<div class="form-group">
  <label for="loginPassword">Password</label>
  <input type="password" id="loginPassword" name="password" class="form-input"
    placeholder="Digite sua senha" required>

```

Figura 60 – Excerto do código que corresponde ao formulário de Login

3.3.3. PÁGINA DE REGISTO DE CONTA

Na criação de contas, foi implementado um formulário que recolhe os dados principais do utilizador, como nome, email, telefone, palavra-passe e tipo de conta. Esta etapa foi importante porque permitiu distinguir logo à partida entre os dois perfis disponíveis no sistema: visitante e lojista. O tipo de administrador foi excluído do formulário por motivos de segurança.

Criar Conta

NOME DE UTILIZADOR

EMAIL

TELEMOVEL

PASSWORD

TIPO DE CONTA

CONTINUAR

[Já tem conta? Fazer Login](#)

ComércioLocal

Juntar-se à comunidade

Crie a sua conta agora. Tenha uma loja ou simplesmente uma conta pessoal.

Figura 61 - página com o formulário de registo

```

<form action="{{ route('register') }}" method="POST" class="form">
  @csrf
  <h1>Criar Conta</h1>

  @if ($errors->has('register'))
  <div class="error-message" style="display: block; margin-bottom: 14px; white-space: pre-wrap;">
    {{ $errors->first('register') }}
  </div>
  @endif

  @if ($errors->any())
  <div class="error-message" style="display: block; margin-bottom: 14px;">
    <strong>Não foi possível concluir o registo.</strong>
    <ul style="margin: 8px 0 0 18px; padding: 0;">
      @foreach ($errors->all() as $error)
      <li>{{ $error }}</li>
      @endforeach
    </ul>
  </div>

```

Figura 62 - Excerto do código que corresponde ao formulário de Registo

3.3.4. VALIDAÇÃO E CRIAÇÃO DA CONTA NO CONTROLADOR

No backend, o processo de registo foi tratado no controlador de autenticação. Aqui são validados os dados inseridos no formulário, é criada a conta na base de dados e a password é guardada de forma segura através de encriptação. Depois da criação, o sistema redireciona o utilizador conforme o tipo de conta escolhido.

```

public function register(Request $request)
{
    try {
        // Prepara o telefone: se vazio, define como null (evita va
        $telefone = trim((string) $request->input('telefone', ''));
        $request->merge([
            'telefone' => $telefone === '' ? null : $telefone,
        ]);
    }

```

Figura 63 – primeiro excerto do controlador com a lógica de registo

3.3.5. DIFERENÇA ENTRE VISITANTE E LOJISTA APÓS O REGISTO

Após criar a conta, o sistema trata de forma diferente os dois tipos de utilizador. Se a conta for de lojista, o utilizador é encaminhado para o registo da loja, para completar os dados do seu negócio. Se for visitante, é redirecionado diretamente para a área principal da aplicação. Esta lógica permitiu adaptar o fluxo de utilização a cada perfil.

```
// Obtém o ID do tipo "lojista" para verificação
$tipoLojistaId = TipoUtilizador::whereRaw('LOWER(nome) = ?', ['lojista'])->value('id');

// Se for lojista, redireciona para o registo da loja
if ($tipoLojistaId && (int) $utilizador->tipo_utilizador_id === (int) $tipoLojistaId) {
    return redirect()->route('loja.registo');
}

// Caso contrário (visitante), vai para a página de explorar
return redirect()->route('explorar');
```

Figura 64 - excerto da lógica de redirecionamento após o registo

3.3.6. PROCESSO DE LOGIN E REDIRECIONAMENTO POR TIPO DE UTILIZADOR

O login foi implementado de forma a validar primeiro as credenciais e, em seguida, autenticar o utilizador através do sistema de sessões do Laravel. Depois da autenticação, o sistema identifica o tipo de conta e faz o respetivo encaminhamento: administrador para o painel de gestão, lojista para os fluxos associados à loja e utilizador normal para a área pública.

```
public function login(Request $request)
{
    // Validar credenciais
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ]);

    // Verificar se a conta existe (mensagem específica para email)
    $accountExists = Utilizador::where('email', $credentials['email'])->exists();

    if (!$accountExists) {
        return back()->withErrors([
            'login' => 'O email não existe.',
        ]->onlyInput('email'));
    }
}
```

Figura 65 – Excerto do código da lógica principal de login

3.4. CRIAÇÃO DA LOJA

Depois de o utilizador se registar como lojista, o sistema encaminha-o para o formulário de criação da loja. Nesta etapa, o acesso é controlado por rotas protegidas e middleware, garantindo que apenas utilizadores com perfil de lojista consigam avançar para este passo. Também é nesta fase que o sistema disponibiliza o formulário de registo da loja para continuar o processo.

Figura 66 - página com o formulário de registo de Loja

```
Route::get('/loja/novo', [LojaController::class, 'registoView'])
    ->middleware(['auth', EnsureLojistaHasLoja::class, IsLojista::class])
    ->name('loja.registo');

Route::post('/loja', [LojaController::class, 'store'])
    ->middleware(['auth', EnsureLojistaHasLoja::class, IsLojista::class])
    ->name('loja.store');
```

Figura 67 - rotas protegidas

No formulário de registo, o lojista introduz os dados principais do seu negócio, como o nome da loja, a categoria, a biografia, a localização, o horário, o email e o telefone. A interface também inclui validação dos campos e mensagens de erro, para garantir que os dados são preenchidos corretamente antes de serem enviados para o sistema.

No controlador, o sistema volta a validar os dados, verifica se o utilizador já tem uma loja associada e, caso esteja tudo correto, cria o registo na base de dados. A loja é criada com estado inicial "pendente", para que o administrador possa analisar e aprovar o pedido mais tarde. No final, o sistema redireciona o lojista para o passo seguinte do processo, que é o upload da foto de perfil da loja.

```
if (Loja::where('id_utilizador', Auth::id())->exists()) {
    return back()->withErrors([
        'nome' => 'Este utilizador ja tem uma loja associada.',
    ])->withInput();
}
```

Figura 68 – Excerto da lógica de criação da loja no controlador

3.5. UPLOAD DA FOTO DE PERFIL DA LOJA

Depois de criar a loja, o lojista passa para o último passo do registo: o carregamento da foto de perfil da loja. Esta funcionalidade foi protegida por rotas e middleware, para garantir que apenas utilizadores com perfil de lojista conseguem aceder a este fluxo. Assim, o sistema encaminha o utilizador para o ecrã apropriado logo após concluir o registo da loja.



Figura 69 - página com a zona para inserir uma fotografia para o perfil da Loja

```
Route::get('/loja/foto', [LojaController::class, 'uploadFotoView'])
->middleware(['auth', EnsureLojistaHasLoja::class, IsLojista::class])
->name('loja.foto.upload');

Route::post('/loja/foto', [LojaController::class, 'storeFotoPerfil'])
->middleware(['auth', EnsureLojistaHasLoja::class, IsLojista::class])
->name('loja.foto.store');
```

Figura 70 - excerto das rotas do upload da foto de perfil

Na interface de upload, o utilizador carrega uma imagem da loja, podendo depois ajustá-la antes de confirmar o envio. A página inclui a área de carregamento, a pré-visualização, os controlos de zoom e a opção de guardar a fotografia final. Esta etapa foi importante para permitir ao lojista personalizar a presença visual da sua loja na plataforma.

No controlador, o sistema recebe a imagem já recortada em base64, valida o formato e o tamanho, remove uma foto anterior caso exista, guarda o novo ficheiro no disco e atualiza o campo da loja na base de dados. No final, termina a sessão por segurança e redireciona o utilizador para o login, informando que a foto foi guardada com sucesso.

```

public function storeFotoPerfil(Request $request)
{
    $request->validate([
        'imagem_recortada' => 'required|string',
    ]);

    $loja = Loja::where('id_utilizador', Auth::id())->first();

    if (!$loja) {
        return redirect()->route('loja.registo');
    }

    $imagemRecortada = (string) $request->input('imagem_recortada');

    // Valida o formato da imagem (deve ser PNG, JPEG ou JPG em base64)
    if (!preg_match('/^data:image\/(png|jpeg|jpg);base64,/', $imagemRecortada)) {
        throw ValidationException::withMessages([
            'imagem_recortada' => 'Formato de imagem invalido.',
        ]);
    }
}

```

Figura 71 - Excerto do controlador com o processamento e gravação da foto

3.6. APROVAÇÃO DA LOJA PELO ADMINISTRADOR

Depois de o lojista concluir o registo da conta e da loja, o pedido fica num estado pendente até ser analisado pelo administrador. Nesta fase, foi implementada uma área específica no painel administrativo para consultar os pedidos de loja e decidir se a conta deve ser aceite ou rejeitada. Esta separação foi importante para garantir controlo manual sobre quem pode começar a utilizar a plataforma como lojista.

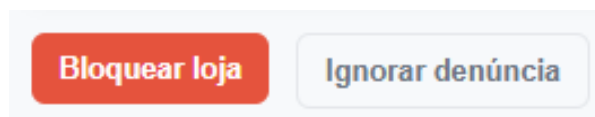


Figura 72 - página com os dois botões de decisão das denúncias

```
<button type="submit" class="btn btn-approve">Aceitar Loja</button>
</form>

<form action="{{ route('admin.pedidos-pendentes.rejeitar', $loja) }}" method="POST"
onsubmit="return confirm('Tem a certeza que quer rejeitar esta loja?');">
  @csrf
  @method('DELETE')
  <button type="submit" class="btn btn-danger">Rejeitar Loja</button>
```

Figura 73 – código dos botões de decisão da secção de pedidos pendentes no painel do administrador

Na lógica de *backend*, foi criada a rota que trata a aprovação e a rejeição do pedido de loja. Quando o administrador aceita o pedido, o estado da loja passa para ativa, permitindo ao lojista iniciar sessão normalmente. Quando rejeita, o sistema marca a loja para eliminação futura, para que a conta e os dados associados sejam removidos no próximo acesso do utilizador.

```
Route::patch('/pedidos-pendentes/{loja}/aceitar', [LojaController::class, 'aprovarPedidoPendente'])
->name('admin.pedidos-pendentes.aprovar');

Route::delete('/pedidos-pendentes/{loja}/rejeitar', [LojaController::class, 'rejeitarPedidoPendente'])
->name('admin.pedidos-pendentes.rejeitar');
```

Figura 74 - rotas de aprovação e rejeição da loja

No controlador, foi implementada a ação de aprovação, que altera o estado da loja para ativa, e a ação de rejeição, que define a loja como bloqueada e marca-a para eliminação. Esta lógica permite manter a base de dados organizada e, ao mesmo tempo, dar ao administrador controlo total sobre os pedidos pendentes.

```

public function aprovarPedidoPendente(Loja $loja)
{
    $estadoAtivaId = EstadoLoja::whereRaw('LOWER(nome) = ?', ['ativa'])->value('id_estado_loja');

    if (!$estadoAtivaId) {
        return back()->withErrors([
            'nome' => 'Nao foi possivel identificar o estado ativo da loja.',
        ]);
    }

    $loja->id_estado_loja = $estadoAtivaId;
    $loja->save();

    return redirect()->route('admin.detalhes-pedido-pendente', $loja)->with('success', 'Loja aprovada com sucesso.');
```

```

}

public function rejeitarPedidoPendente(Loja $loja)
{
    $estadoBloqueadaId = EstadoLoja::whereRaw('LOWER(nome) = ?', ['bloqueada'])->value('id_estado_loja');

    if (!$estadoBloqueadaId) {
        return back()->withErrors([
            'nome' => 'Nao foi possivel identificar o estado bloqueado da loja.',
        ]);
    }

    $loja->id_estado_loja = $estadoBloqueadaId;
    $loja->loja_por_eliminar = true; // Marca para eliminação futura
    $loja->save();

    return redirect()->route('admin.pedidos-pendentes')->with('success', 'Loja rejeitada. Os dados serao eliminados no proximo login do lojista.');
```

```

}

```

Figura 75 - código do controlador com a aprovação e rejeição da loja

3.7. GESTÃO DE PUBLICAÇÕES POR PARTE DO LOJISTA

A gestão de publicações foi implementada na área privada do lojista, dentro do seu perfil, permitindo-lhe criar e organizar conteúdo para a sua loja. Nesta secção, o lojista pode publicar texto, adicionar imagens ou inserir vídeos através de um link público, o que torna a plataforma mais flexível e evita o armazenamento desnecessário de ficheiros pesados no servidor. A interface foi desenhada para simplificar o processo de publicação e orientar o utilizador de forma clara durante a criação do conteúdo.

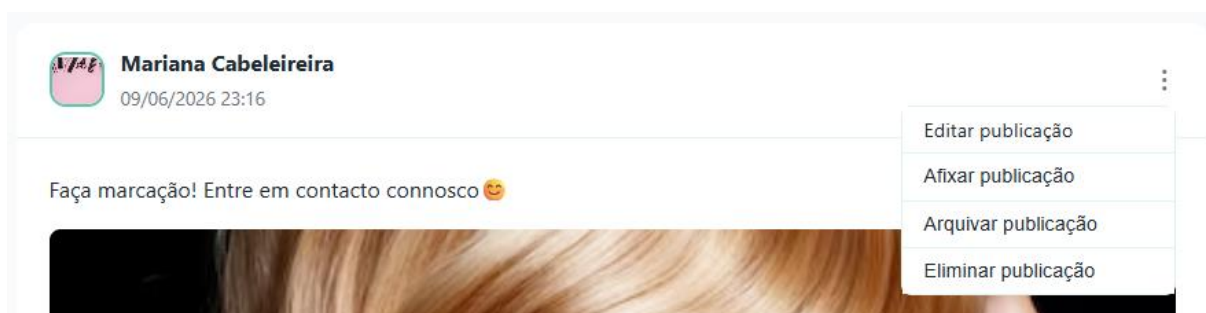


Figura 76 - página com as funcionalidades para gerir as publicações

Nas publicações já criadas, o lojista tem acesso a um menu de opções em cada publicação, onde pode editar o conteúdo, afixar a publicação no topo do perfil, arquivá-la ou eliminá-la. Esta funcionalidade foi pensada para dar controlo total sobre o conteúdo publicado, permitindo ao lojista destacar informação importante e gerir publicações antigas de forma organizada.

Além da interface, foi desenvolvida a lógica no controlador responsável por criar e gerir as publicações. O sistema valida os dados introduzidos, identifica o tipo de publicação, guarda corretamente a imagem ou o link do vídeo e associa a publicação à loja do lojista. Depois disso, permite ainda alterar a publicação, arquivá-la, afixá-la ou desarquivá-la conforme o estado em que se encontra.

```
public function destroy(int $idPublicacao)
{
    $loja = Loja::where('id_utilizador', Auth::id())->first();

    if (!$loja) {
        return redirect()->route('loja.registo');
    }

    $publicacao = Publicacao::where('id_publicacao', $idPublicacao)->firstOrFail();

    if ((int) $publicacao->id_loja !== (int) $loja->id_loja) {
        abort(403);
    }

    // Elimina o ficheiro físico (se for imagem local)
    $this->deleteStoredFile($publicacao->ficheiro);

    // Elimina o registo na base de dados
    $publicacao->delete();

    return redirect()->route('perfil.lojista')->with('success', 'Publicacao eliminada com sucesso.');
```

Figura 77 - código do controlador para eliminar uma publicação

3.8. DENÚNCIAS POR PARTE DO UTILIZADOR VISITANTE

Foi implementada uma funcionalidade que permite denunciar lojas ou publicações consideradas inadequadas. O utilizador acede a uma página própria de denúncia, onde pode ver primeiro o conteúdo que está a denunciar e depois preencher um formulário com o motivo e a descrição detalhada. Esta abordagem facilita a moderação da plataforma e ajuda a manter a comunidade mais segura.

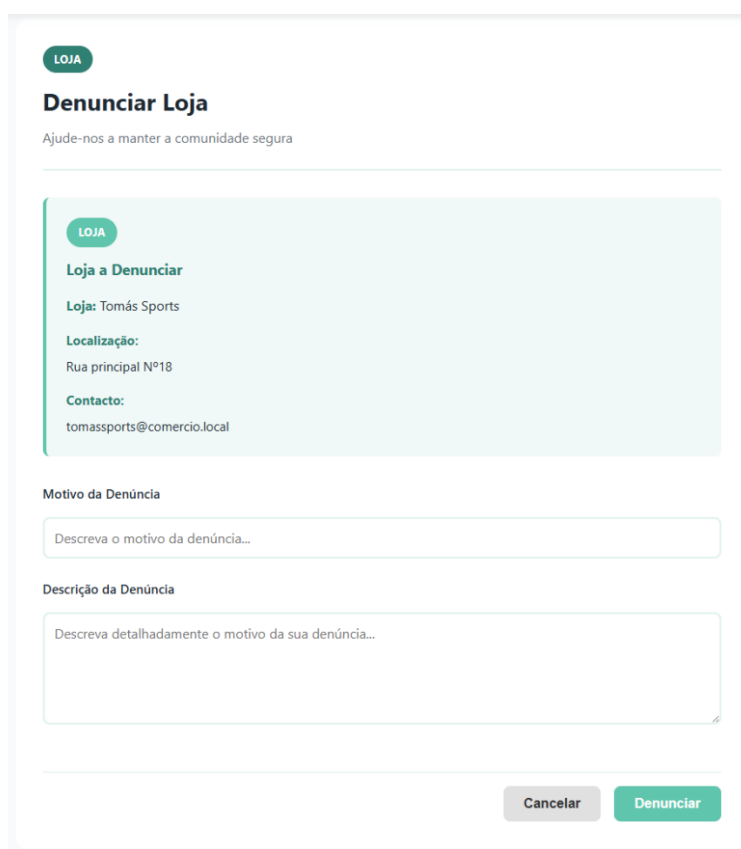


Figura 78 - página com o formulário de denúncia de uma Loja

```
Route::get('/denunciar/publicacao/{loja}/{publicacao}', function (Loja $loja, Publicacao $publicacao) {
    return view('denuncias.criar', [
        'loja' => $loja,
        'publicacao' => $publicacao,
        'tipo' => 'publicacao',
    ]);
});
->middleware(['auth', EnsureLojistaHasLoja::class])
->name('denuncias.publicacao.criar');

Route::get('/denunciar/loja/{loja}', function (Loja $loja) {
    return view('denuncias.criar', [
        'loja' => $loja,
        'tipo' => 'loja',
    ]);
});
->middleware(['auth', EnsureLojistaHasLoja::class])
->name('denuncias.loja.criar');

Route::post('/denuncias', [DenunciaController::class, 'store'])
->middleware(['auth', EnsureLojistaHasLoja::class])
->name('denuncias.store');
```

Figura 79 - rotas de funcionalidade de denúncias

Na interface de denúncia, é apresentado o item selecionado, seja uma loja ou uma publicação, para que o utilizador confirme exatamente o que está a reportar. A página inclui também o formulário com os campos necessários para submeter a denúncia, bem como a lógica de envio assíncrono, que melhora a experiência de utilização e apresenta feedback imediato ao utilizador.

Depois de submetida a denúncia, o sistema valida os dados recebidos, confirma o tipo de denúncia, associa o estado inicial como pendente e guarda o registo na base de dados. A denúncia fica assim registada para posterior análise administrativa, garantindo que o processo de moderação fica centralizado e controlado.

```
public function store(Request $request)
{
    try {
        $validated = $request->validate([
            'tipo' => 'required|in:publicacao,loja',
            'motivo' => 'required|string',
            'descricao' => 'required|string|min:10',
            'id_publicacao' => 'nullable|integer|exists:publicacao,id_publicacao',
            'id_loja' => 'nullable|integer|exists:loja,id_loja',
        ]);
    }
}
```

Figura 80 – Excerto do código do controlador para registar uma denúncia

3.9. EXPLORAR LOJAS E PUBLICAÇÕES

A página Explorar permite a utilizadores autenticados pesquisar e filtrar lojas ativas por nome, categoria e ordenação ($A \rightarrow Z$ / $Z \rightarrow A$), excluindo lojas que o próprio utilizador marcou como "sem interesse". O método explorar constrói a consulta aplicando os filtros e ordenação e envia lojas e categorias para a vista; a rota exige auth e o middleware EnsureLojistaHasLoja. A vista apresenta o formulário de filtros (pesquisa, categoria, ordenação) e uma grelha de cartões com imagem, categoria, descrição e ações (ver loja, marcar sem interesse).

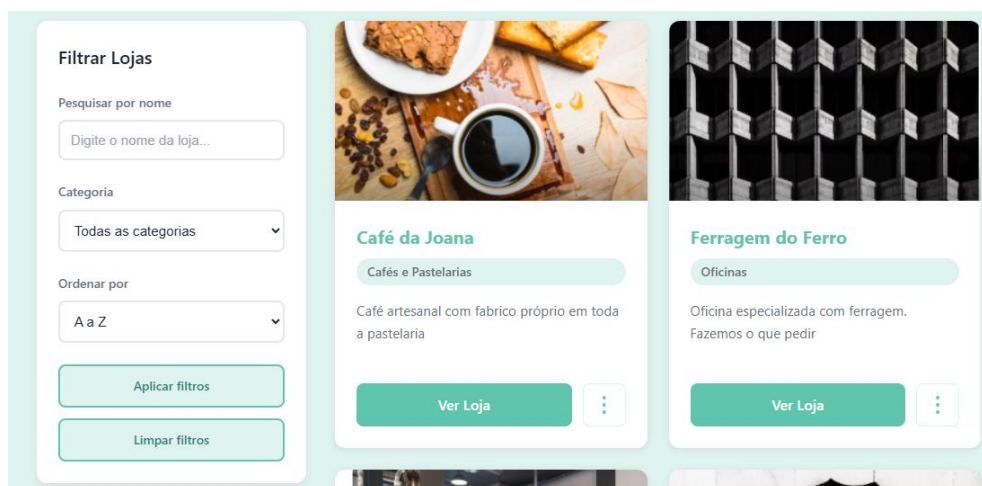


Figura 81 - Página com o formulário de filtros e a zona que gera os cartões das Lojas

```
public function explorar(Request $request)
{
    $pesquisa = trim((string) $request->query('pesquisa', ''));
    $categoriaId = $request->query('categoria');
    $ordenacao = (string) $request->query('ordenacao', 'az');

    if (!in_array($ordenacao, ['az', 'za'], true)) {
        $ordenacao = 'az';
    }

    $filtrosAplicados = $pesquisa !== '' || !empty($categoriaId) || $ordenacao === 'za';
}
```

Figura 82 – Excerto do método que aplica filtros, ordenação e retorna a vista

4. PROBLEMAS E SOLUÇÕES ENCONTRADAS

O presente capítulo tem como objetivo documentar os principais problemas e obstáculos enfrentados ao longo do desenvolvimento do projeto ComércioLocal, bem como as soluções implementadas para os ultrapassar. A identificação e resolução destas dificuldades constituem uma parte fundamental do processo de desenvolvimento, demonstrando a capacidade de adaptação e de resolução de problemas por parte do programador.

4.1. GESTÃO DE ARMAZENAMENTO PARA PUBLICAÇÕES DE VÍDEO

Durante o desenvolvimento da funcionalidade que permite aos lojistas criarem publicações multimédia nas suas lojas, surgiu um problema de escalabilidade relacionado com o armazenamento de ficheiros de vídeo no servidor local. Numa primeira abordagem, a permissão de upload direto de vídeos para o servidor implicaria um crescimento rápido do espaço ocupado em disco, uma vez que os ficheiros de vídeo tendem a ter dimensões significativamente superiores às imagens. Adicionalmente, esta abordagem acarretaria uma maior carga no servidor, maior complexidade de manutenção (nomeadamente ao nível de backups, transferência e eliminação de ficheiros pesados) e custos acrescidos com armazenamento.

Para resolver este problema, foi adotada uma estratégia de publicação por ligação externa. Em vez de guardar o ficheiro de vídeo localmente no servidor da aplicação, a plataforma passou a armazenar apenas o URL público do vídeo, ou seja, uma ligação para plataformas como o YouTube, Vimeo ou para ficheiros alojados em serviços de terceiros, como o Pixabay. Desta forma, os vídeos continuam disponíveis para visualização por parte do utilizador final, mas sem sobrecarregar o armazenamento do sistema, uma vez que o ficheiro físico permanece alojado no servidor de origem.

A implementação desta solução foi realizada com uma validação específica do campo de URL para vídeos, bem como com a separação de comportamento entre o tratamento de imagens e o tratamento de vídeos. As imagens continuaram a ser armazenadas localmente no servidor, uma vez que os seus ficheiros são geralmente mais leves e mais fáceis de gerir, enquanto os vídeos passaram a ser guardados exclusivamente como ligações externas, associadas a um URL fornecido pelo lojista. Na apresentação dos conteúdos, a aplicação deteta o tipo de ligação fornecida e tenta renderizar o vídeo de forma adequada, utilizando o modo embed quando possível (por exemplo, para vídeos do YouTube ou Vimeo) ou a reprodução direta por URL quando se trata de um ficheiro com extensão .mp4 ou similar.

A Figura 81 ilustra o botão desenvolvido para permitir ao lojista escolher a opção de adicionar um vídeo através de uma ligação externa, enquanto a Figura 82 apresenta a zona de input onde o utilizador insere o URL do vídeo desejado. A validação do campo foi implementada com a regra `video_url => nullable|url|max:2048|required_if:type_media_video`, conforme se pode observar na

Figura 83, que garante que o URL fornecido é válido, não excede o comprimento máximo permitido e é obrigatório quando o tipo de publicação é vídeo.

```
<span>
  
  Vídeo por link
</span>
```

Figura 83 - Botão para escolher a opção de vídeo por ligação externa

```
<input id="video_url" name="video_url" type="url" class="post-url-input"
  value="{{ old('video_url') }}"
  placeholder="Cole aqui o link público do vídeo, por exemplo https://www.youtube.com/watch?v=...">
```

Figura 84 - Zona de input para inserir o URL do vídeo

```
$caminhoFicheiro = null;
if ($tipoNome === 'video') {
  $caminhoFicheiro = $request->input('video_url');
```

Figura 85 - Código de validação do campo de URL para vídeo

Relativamente à apresentação dos vídeos, a plataforma adota um comportamento dinâmico consoante a origem da ligação fornecida. Quando o URL corresponde a uma plataforma como o YouTube ou o Vimeo, a aplicação incorpora o vídeo utilizando o player nativo da plataforma, proporcionando uma experiência de visualização familiar ao utilizador, como se pode verificar na Figura 85 (para o YouTube) e na Figura 87, que ilustra igualmente a reprodução com o player do YouTube. Por outro lado, quando o URL fornecido aponta diretamente para um ficheiro de vídeo (por exemplo, com extensão .mp4, .webm ou .ogg), a aplicação utiliza o player de vídeo nativo do navegador, que permite a reprodução direta do conteúdo, conforme ilustrado na Figura 86.

```
<iframe src="{{ $videoEmbedUrl }}" class="post-media post-video-embed" title="Vídeo da publicação" loading="lazy"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share"
  allowfullscreen></iframe>
```

Figura 86 - Vídeo incorporado com o player do YouTube

```
<video class="post-media post-video-player" controls playsinline preload="metadata">
  <source src="{{ $directVideoSource }}">
  O seu navegador não suporta reprodução de vídeo.
</video>
```

Figura 87 - Vídeo reproduzido com o player nativo do navegador (URL com extensão .mp4)

```
if (preg_match('/(?:youtu.be|youtube.com)/(?:watch?v=|embed\/))([A-Za-z0-9_-]{6,})/i', $url, $matches)) {
  return 'https://www.youtube.com/embed/' . $matches[1];
}
```

Figura 88 - Exemplo de vídeo do YouTube com o player da plataforma

Com esta solução, foi possível reduzir significativamente o consumo de espaço de armazenamento local, melhorar a performance geral da aplicação (uma vez que o servidor não precisa de processar nem servir ficheiros de vídeo de grandes dimensões) e tornar o sistema mais

sustentável e preparado para um crescimento futuro, uma vez que a escalabilidade do armazenamento passa a ser gerida por serviços externos especializados.

4.2. EXPERIÊNCIA DE LOGIN DO LOJISTA APÓS REJEIÇÃO DO PEDIDO DE CONTA

Durante o desenvolvimento do módulo de autenticação e gestão de utilizadores, foi identificado um problema relacionado com a experiência de utilizador no fluxo de registo e validação de contas de lojistas.

Quando um lojista criava uma conta na plataforma, esta ficava em estado pendente até que o administrador analisasse e tomasse uma decisão sobre o pedido. Caso o pedido fosse rejeitado, os dados do utilizador e da loja eram eliminados da base de dados, de modo a manter a otimização do armazenamento e a evitar a acumulação de registos não aprovados. No entanto, sem qualquer tratamento adicional, quando o utilizador tentava iniciar sessão novamente, o sistema apresentava apenas uma mensagem genérica indicando que o endereço de correio eletrónico não existia na base de dados, o que não explicava o motivo real da impossibilidade de autenticação e gerava confusão e frustração no utilizador, uma vez que este desconhecia que o seu pedido havia sido rejeitado.

Para resolver este problema, foi implementada uma lógica específica no processo de autenticação, que passou a distinguir entre a ausência de conta e a rejeição prévia do pedido. O fluxo adotado funciona da seguinte forma: quando o administrador rejeita um pedido de registo de uma loja, o sistema não elimina imediatamente os dados. Em vez disso, marca a loja com um indicador booleano (`loja_por_eliminar = true`) e altera o seu estado para "bloqueada", preservando temporariamente os dados na base de dados. Esta abordagem permite que, no próximo login desse lojista, o sistema detete a existência dessa marca e apresente uma mensagem explícita a informar que a conta não foi aceite, explicando o motivo pelo qual o acesso não é permitido. De seguida, o sistema elimina os dados registados (loja e utilizador) de forma controlada, dentro de uma transação que garante a integridade da base de dados, e termina a sessão do utilizador, redirecionando-o para a página de login com a mensagem de erro adequada.

A implementação desta solução envolveu várias componentes do sistema. A Figura 87 ilustra o botão disponível no painel administrativo que permite ao administrador rejeitar o pedido de uma loja pendente, acionando o processo de marcação para eliminação futura.

```
<form action="{{ route('admin.pedidos-pendentes.rejeitar', $loja) }}" method="POST"
  onsubmit="return confirm('Tem a certeza que quer rejeitar esta loja?');">
  @csrf
  @method('DELETE')
  <button type="submit" class="btn btn-danger">Rejeitar Loja</button>
</form>
```

Figura 89 - Botão para rejeitar a loja no painel administrativo

A Figura 88 apresenta a função `rejeitarPedidoPendente`, que é executada quando o administrador rejeita um pedido. Nesta função, o sistema identifica o estado "bloqueada" na tabela

estado_loja, atribui esse estado à loja em questão e define o campo loja_por_eliminar como verdadeiro, guardando as alterações na base de dados. Desta forma, a loja fica marcada para eliminação, mantendo-se os dados até ao próximo login do lojista. A função redireciona o administrador de volta para a lista de pedidos pendentes, exibindo uma mensagem de sucesso a informar que a loja foi rejeitada e que os dados serão eliminados no próximo login do lojista.

```
public function rejeitarPedidoPendente(Loja $loja)
{
    $estadoBloqueadaId = EstadoLoja::whereRaw('LOWER(nome) = ?', ['bloqueada'])->value('id_estado_loja');

    if (!$estadoBloqueadaId) {
        return back()->withErrors([
            'nome' => 'Nao foi possivel identificar o estado bloqueado da loja.',
        ]);
    }

    $loja->id_estado_loja = $estadoBloqueadaId;
    $loja->loja_por_eliminar = true;
    $loja->save();

    return redirect()->route('admin.pedidos-pendentes')->with('success', 'Loja rejeitada. Os dados serao eliminados no proximo login do lojista.');
```

Figura 90 - Função para rejeitar o pedido e marcar a loja para eliminação

A Figura 89 contém a lógica implementada no processo de autenticação, que deteta se o utilizador que está a tentar iniciar sessão possui uma loja marcada com o indicador loja_por_eliminar. Caso essa condição seja verdadeira, o sistema constrói uma mensagem explicativa, informando o utilizador de que a sua loja não foi aceite no sistema e que, por questões de otimização da base de dados, todos os dados registados serão eliminados. De seguida, executa uma transação que elimina a loja e o utilizador associado, efetua o logout forçado do utilizador, invalida a sessão e regenera o token de segurança, redirecionando o utilizador para a página de login com a mensagem de erro personalizada.

```
if ((bool) $loja->loja_por_eliminar) {
    $mensagemRejeicao = 'A tua loja não foi aceite no sistema. Para maior otimização técnica (base de dados), vamos eliminar todos os dados que foram registados na tua tentativa de registo.';

    DB::transaction(function () use ($loja, $user) {
        if (!empty($loja->foto_perfil)) {
            Storage::disk('public')->delete($loja->foto_perfil);
        }

        $loja->delete();
        Utilizador::where('id', $user->id)->delete();
    });

    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect()->route('login')
        ->withErrors([
            'login' => $mensagemRejeicao,
        ])->withInput($request->only('email'));
```

Figura 91 - Detecção do indicador de eliminação e apresentação da mensagem ao lojista

Por fim, a Figura 90 ilustra o código frontend que exibe a mensagem de erro na página de login, utilizando o mecanismo de flashes de erro do Laravel para apresentar a mensagem personalizada de forma clara e visível ao utilizador.

```
@if ($errors->has('login'))
<div class="error-message" style="display: block; margin-bottom: 14px; white-space: pre-wrap;">
  {{ $errors->first('login') }}
</div>
@endif
```

Figura 92 - Exibição da mensagem de erro na página de login

Com esta abordagem, foi possível melhorar significativamente a clareza da comunicação com o utilizador, evitando mensagens ambíguas e garantindo que o lojista compreende a razão pela qual não consegue aceder à plataforma. Simultaneamente, a otimização técnica da base de dados é mantida, uma vez que os dados são eliminados de forma controlada no momento adequado, sem comprometer a integridade do sistema ou a experiência do utilizador.

4.3. ERRO DE SINTAXE NA PÁGINA ADMINISTRATIVA DE GESTÃO DE LOJAS

Durante a implementação da página administrativa destinada à gestão de lojas, surgiu um erro de sintaxe que impedia o carregamento correto da interface, impossibilitando o administrador de visualizar e gerir os estabelecimentos registados na plataforma. Este tipo de erro, embora comum em fases de desenvolvimento, requer uma análise cuidada para identificar a sua origem e aplicar a correção adequada.

A Figura 109 ilustra a mensagem de erro apresentada pelo Laravel, que, através do seu sistema de depuração (debug), fornece informações detalhadas sobre a localização e a natureza do problema. Neste caso, o erro foi detetado numa view Blade, indicando que a origem do problema se encontrava na camada de apresentação da aplicação.

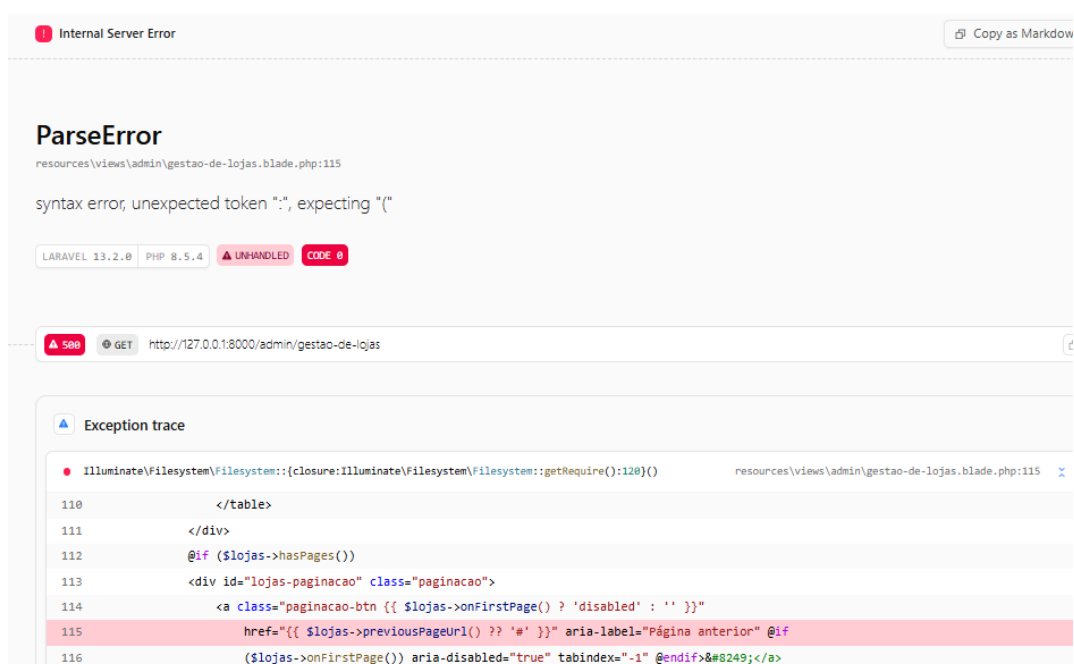


Figura 94 - Código Blade que estava a causar o erro de sintaxe

A solução para este problema passou pela reorganização do código Blade, reescrevendo a lógica de paginação de forma mais clara e estruturada. Foram separadas as condições, garantindo que cada bloco condicional fosse devidamente aberto e fechado, e a indentação foi ajustada para refletir corretamente a hierarquia do código. A Figura 111 apresenta o código corrigido, onde a estrutura se encontra mais legível e organizada, com as condições devidamente aninhadas e os elementos de paginação a serem exibidos apenas quando existem dados para apresentar.

```
@if ($lojas->onFirstPage())
<span class="paginacao-btn disabled" aria-label="Página anterior" aria-disabled="true"
tabindex="-1">
    &#8249;
</span>
@else
<a class="paginacao-btn" href="{{ $lojas->previousPageUrl() }}" aria-label="Página anterior">
    &#8249;
</a>
@endif
```

Figura 95 - Código Blade corrigido que solucionou o problema

Com esta correção, a página de gestão de lojas passou a ser carregada corretamente, sem erros de sintaxe, permitindo ao administrador visualizar a lista de estabelecimentos, aplicar filtros de pesquisa e aceder às funcionalidades de gestão, como a aprovação, bloqueio ou rejeição de lojas. Este problema realçou a importância de uma codificação cuidada e estruturada, especialmente em views que combinam lógica de apresentação com elementos dinâmicos, bem como a utilidade das mensagens de erro do Laravel na rápida identificação e resolução de problemas.

4.4. CONTROLLERS COM MUITO CÓDIGO

Ao longo do desenvolvimento do projeto, e à medida que novas funcionalidades iam sendo implementadas, surgiu um problema que, embora não afetasse o funcionamento da aplicação, comprometia a legibilidade, a manutenibilidade e a clareza do código-fonte. Este problema consistia no crescimento excessivo dos ficheiros dos controladores (controllers), que acumulavam um número cada vez maior de métodos e linhas de código, tornando difícil a compreensão da lógica implementada e dificultando futuras alterações ou correções.

A Figura 112 ilustra este cenário, evidenciando a extensão do código num único ficheiro, onde múltiplas funcionalidades se encontravam concentradas. Esta abordagem, embora funcional, violava o princípio da responsabilidade única (Single Responsibility Principle), que preconiza que cada classe deve ter uma única razão para mudar, ou seja, deve ser responsável por uma única funcionalidade ou conjunto de funcionalidades coesas.

```
359         return view('p
360     }
```

Figura 96 - Exemplo da dimensão elevada de um ficheiro de controlador

A solução adotada revelou-se simples e eficaz, baseando-se na reorganização estrutural dos controladores. Em vez de manter todas as funcionalidades num único ficheiro, procedeu-se à criação de subpastas dentro da pasta Controllers, organizadas por domínios funcionais da aplicação. Dentro de cada pasta, foram criados novos ficheiros de controladores, cada um dedicado a uma funcionalidade específica ou a um conjunto de funcionalidades intimamente relacionadas.

A Figura 113 exemplifica esta reorganização, mostrando a pasta Loja, que passou a conter vários ficheiros de controladores, cada um responsável por uma área distinta da gestão das lojas. Desta forma, o código tornou-se mais modular, mais fácil de ler, de interpretar e de manter, uma vez que cada ficheiro passou a ter uma dimensão mais reduzida e um propósito claramente definido. Esta abordagem facilita, também, a localização de métodos específicos e a identificação de possíveis erros, reduzindo o tempo necessário para depuração e manutenção.

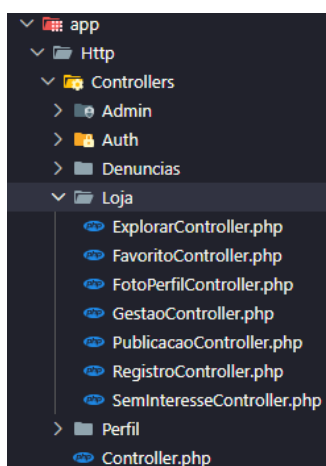


Figura 97 - Exemplo da organização dos controladores da pasta Loja

Com esta solução, foi possível melhorar significativamente a qualidade do código, promovendo uma estrutura mais limpa, organizada e alinhada com as boas práticas de desenvolvimento de software, sem comprometer o funcionamento ou o desempenho da aplicação.

5. CONCLUSÃO

5.1. CUMPRIMENTO DOS OBJETIVOS

O projeto ComércioLocal foi concluído com sucesso, tendo sido desenvolvido de acordo com os objetivos e as ideias iniciais que orientaram a sua conceção. As funcionalidades essenciais previstas na fase de planeamento foram implementadas e encontram-se operacionais, nomeadamente a gestão de utilizadores e autenticação, o registo e aprovação de lojas, a criação de publicações multimédia, o sistema de favoritos e o painel administrativo para moderação de conteúdos.

Desta forma, considera-se que o projeto cumpriu integralmente os objetivos definidos na introdução do relatório, designadamente a centralização da informação do comércio local, o aumento da visibilidade digital dos pequenos negócios, a facilitação da descoberta de estabelecimentos por parte dos consumidores e a promoção da economia local e do comércio de proximidade no concelho das Caldas da Rainha.

5.2. PERSPETIVAS DE EVOLUÇÃO FUTURA

Embora o projeto, na sua versão atual, se encontre finalizado e operacional, é possível identificar um conjunto de funcionalidades que poderiam ser adicionadas numa eventual evolução futura, ampliando o valor e o alcance da plataforma. Ainda que não esteja prevista a continuidade do desenvolvimento, estas ideias refletem o potencial de crescimento do projeto.

Nesse sentido, as principais funcionalidades que poderiam ser integradas numa versão futura seriam, por um lado, a implementação de um sistema de conversação, através de chats que permitissem a comunicação direta entre visitantes e comerciantes, facilitando o esclarecimento de dúvidas e o agendamento de serviços, e, por outro lado, a criação de um sistema de loja online que possibilitasse a realização de encomendas, ou seja, a compra de produtos diretamente ao lojista através da plataforma. Esta última funcionalidade representaria uma evolução significativa, transformando o ComércioLocal de um mero diretório informativo para uma plataforma transacional, com impacto direto nas vendas do comércio local.

5.3. AUTOAVALIAÇÃO

Relativamente à avaliação do meu desempenho pessoal ao longo do desenvolvimento do projeto, considero a minha prestação como positiva. Ao longo de todo o processo, procurei cumprir com rigor as etapas planeadas, superar os desafios técnicos que foram surgindo e garantir que o produto final correspondesse às expectativas e aos requisitos definidos na fase de conceção.

Apesar das dificuldades encontradas, nomeadamente ao nível da gestão de armazenamento de vídeos e da experiência de utilizador no fluxo de registo de lojistas, que foram devidamente documentadas e resolvidas, conforme descrito no capítulo 4, considero que os objetivos foram alcançados e que o projeto reflete, de forma integrada, as competências desenvolvidas ao longo do curso, designadamente ao nível da programação *frontend* e *backend*, da modelação e gestão de bases de dados, do design de interfaces e da gestão de projetos.

Em suma, o ComércioLocal revelou-se um desafio formativo e enriquecedor, que me permitiu consolidar conhecimentos técnicos e desenvolver competências de resolução de problemas, planeamento e autonomia. Considero que o trabalho realizado corresponde ao que foi idealizado no início do projeto, razão pela qual avalio a minha prestação de forma globalmente satisfatória e o projeto como um contributo válido para a valorização do comércio local das Caldas da Rainha.

REFERÊNCIAS BIBLIOGRÁFICAS

As referências bibliográficas devem ser feitas de acordo com as Normas APA – 7.^a edição e ordenam-se por ordem alfabética dos apelidos dos autores.

Documentos na Web

Laravel. (s.d.). Laravel documentation. <https://laravel.com/docs>

MDN Web Docs. (s.d.). CSS: Cascading Style Sheets. <https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs. (s.d.). HTML: HyperText Markup Language. <https://developer.mozilla.org/en-US/docs/Web/HTML>

MDN Web Docs. (s.d.). JavaScript. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

PHP Group. (s.d.). PHP manual. https://www.php.net/manual/pt_BR/

Vídeo do Youtube

Everton Dev. (2023, setembro 15). Manipulando JSON e criar elementos HTML - Criando Card HTML com arquivo JSON usando JavaScript [Vídeo]. YouTube.

https://www.youtube.com/watch?v=H_1Ml4dC8z4

Will Dev. (2022, setembro 20). Criando Barra de Pesquisa com JavaScript [Vídeo]. YouTube.

<https://www.youtube.com/watch?v=3kOm3rcLIGM>

Celke. (2018, abril, 05). Como criar o botão mostrar ou ocultar senha [Vídeo]. YouTube.

<https://www.youtube.com/watch?v=6NlCkOR6os>

IDENTIFICAÇÃO DO ALUNO

NOME COMPLETO: VLADIIMIRO PANKRATAU

N.º DE PROCESSO: 103856

CICLO DE ESTUDOS: ENSINO SECUNDÁRIO

TELEMÓVEL: 912826677

E-MAIL: V.PANKRATAU.P19@GMAIL.COM
